学号____20185311____ 密级_____

东北大学本科毕业论文

一种基于屏幕空间的实时光线追踪方法研究与应用

学 院 名 称：软件学院

专 业 名 称：数字媒体技术

学 生 姓 名：庄正涛

指 导 教 师：高天寒　　教授

高红杰　　工程师

二〇二二年六月

一种基于屏幕空间的实时光线追踪方法研究与应用

作者姓名： 庄正涛

校内指导教师： 高天寒 教授

校外指导教师： 高红杰 工程师

单位名称： 软件学院

专业名称： 数字媒体技术

东 北 大 学

2022 年 6 月

Research and Application of Real-Time Ray Tracing Method based on Screen Space

by Zhuang Zhengtao

Supervisor: Professor Gao Tianhan
Associate Supervisor: Engineer Gao Hongjie

Northeastern University

June 2022

郑 重 声 明

本人呈交的学位论文，是在导师的指导下，独立进行研究工作所取得的成果，所有数据、图片资料真实可靠。尽我所知，除文中已经注明引用的内容外，本学位论文的研究成果不包含他人享有著作权的内容。对本论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确的方式标明。本学位论文的知识产权归属于培养单位。

本人签名：庄正涛　　日期：2022 年 6 月 8 日

摘 要

随着交互式 3D 数字应用的快速普及与发展，用户对画面表现和动态光影效果的要求越来越高。基于光线追踪的全局光照技术是真实感图形渲染中的一项重要技术。经典的光线追踪技术（如路径追踪等），因其计算复杂度过高，并不适用于实时图形渲染应用；近年来，由 NVIDIA 提出的 RTXGI 等实时光线追踪技术则依赖原生硬件支持，且对硬件性能要求较高，进一步提升了用户成本。本文提出采用基于屏幕空间的实时光线追踪方法，降低用户硬件成本，同时满足用户对画面质量的要求。

本文通过在软件层面模拟光线步进，设计并实现了一套基于屏幕空间的实时光线追踪方法。该方法基于渲染方程和蒙特卡洛积分，对屏幕上的每个着色点在半球范围内进行均匀随机采样，在屏幕空间下进行光线步进。并根据深度缓冲计算光线与场景的碰撞信息，然后对 Scene Color 等纹理进行采样，从而计算场景中的间接漫反射光照。针对方法的性能瓶颈，使用了层次 Z 缓冲技术提高光线与场景求交效率；并通过综合应用时域过滤和双边过滤对光线步进的结果进行降噪处理，从而实现高质量的实时全局光照。通过大量真实场景的实时渲染结果表明，本文所提方法具备良好的兼容性，可以对预计算光照中的间接光照进行补充；此外本方法提供了丰富的动态可配置参数，以适应不同硬件性能。

本文首先介绍了所提方法的研究背景、研究意义与国内外现状，并对完成系统所需的相关技术进行详细介绍。而后对系统的功能性需求与非功能性需求做出了全面的分析，对系统流程进行了详细的设计，详述了程序功能和画面实现等方面的内容。对本文方法在 Unity 引擎内置渲染管线中的延迟渲染路径中进行了实现，在经典的 Sponza 场景中对方法进行了测试与应用。同时基于 Unity 引擎优秀的可扩展性，能够快速使用该技术进行应用构建和发布。

关键词：实时全局光照；屏幕空间；光线步进；Hi-Z 缓冲；Unity

ABSTRACT

With the rapid popularization and development of interactive 3D digital applications, users have higher requirements for graphics performance. Global illumination method based on ray tracing is an important technology in realistic graphics rendering. The classical ray tracing technology(such as path tracing, etc.) is not suitable for real-time graphics applications because of its high computational complexity. RTXGI and other real-time ray tracing technologies proposed by NVIDIA in recent years rely on native hardware support and require high hardware performance which further improve the users' hardware cost.The method in this paper can reduce users' hardware cost and meet their requirements for graphics quality on the basis of realizing real-time ray tracing.

In this paper, a real-time ray tracing method based on screen space is designed and implemented by simulating ray marching at the software level. Based on the rendering equation and Monte Carlo integral, the method takes uniform random sampling of each shading point on the screen in the hemispheres and performs ray marching in the screen space. The collision information between ray and scene is calculated according to the depth buffer, and then the indirect diffuse illumination is calculated by sampling the texture such as Scene Color. To solve the performance bottleneck of the method, the hierarchical Z-buffer technology is used to accelerate the intersection of ray and scene. In addition, the noise reduction is achieved by using the temporal filter and bilateral filter. Through a large number of real-time rendering results, the method has good compatibility and can supplement the indirect illumination in the precomputed illumination. This method provides rich dynamic configurable parameters to adapt to different hardware performance scenarios.

In this paper, the research background, significance, current situation of the method and related technologies required to complete the system are introduced. Then make a comprehensive analysis of the functional and non-functional requirements of the system, introduce the detailed design of the system process. The method is implemented in the deferred rendering path in the built-in rendering pipeline of Unity Engine, and tested in a classical Sponza scene. Based on the excellent scalability of Unity Engine, the method can quickly apply this technology for application construction and release.

Key words: Real-time GI; Screen Space; Ray Marching; Hierarchical Z-Buffer; Unity

目 录

摘 要 II
ABSTRACT III
1 绪 论 1
1.1 课题研究背景 1
1.1.1 全局光照技术发展历程 1
1.1.2 混合渲染管线发展历程 2
1.2 课题研究意义 2
1.3 国内外研究现状 3
1.3.1 基于预计算的 GI 技术现状 3
1.3.2 基于屏幕空间的实时 GI 技术现状 4
1.3.3 基于 3D 空间的实时 GI 技术现状 4
1.4 论文研究内容 5
1.5 论文的组织结构 5
2 相关技术 7
2.1 渲染管线 7
2.1.1 渲染管线中的不同阶段 7
2.1.2 渲染管线中的空间变换 8
2.1.3 延迟渲染路径 8
2.2 屏幕空间 GI 技术 10
2.2.1 屏幕空间算法特点 10
2.2.2 环境光遮蔽 10
2.2.3 屏幕空间方向性遮蔽 11
2.2.4 屏幕空间反射 11
2.3 3D 空间 GI 技术 12
2.3.1 光照体积传播算法 12
2.3.2 体素全局光照算法 13
2.4 Unity 引擎 14

2.5 本章小结 14
3 需求分析 15
3.1 业务需求 15
3.2 用户需求 16
3.2.1 高质量实时 GI 效果 16
3.2.2 自定义渲染质量及性能需求 16
3.2.3 Unity 渲染流程及开发工具兼容性 16
3.3 功能性需求 17
3.3.1 屏幕空间中的光线追踪 17
3.3.2 光线追踪的加速优化 17
3.3.3 光线追踪的降噪处理 18
3.3.4 其他优化 19
3.3.5 用户交互模块 19
3.4 非功能性需求 19
3.4.1 性能需求 19
3.4.2 健壮性需求 20
3.4.3 易用性需求 20
3.5 本章小结 20
4 系统设计 21
4.1 系统整体功能设计 21
4.2 数据准备模块设计 21
4.3 光线追踪模块设计 23
4.3.1 渲染方程与蒙特卡洛积分 23
4.3.2 光线追踪系统设计 25
4.4 加速优化模块设计 26
4.5 降噪处理模块设计 28
4.5.1 时域过滤 29
4.5.2 双边过滤 29
4.6 其他优化设计 29

4.6.1 屏幕边缘渐隐 29
4.6.2 半分辨率追踪 29
4.7 用户交互模块设计 30
4.7.1 功能设计 30
4.7.2 界面设计 31
4.8 本章小结 32
5 系统实现 33
5.1 数据准备模块实现 33
5.1.1 参数配置文件 33
5.1.2 数据资源管理 34
5.2 构建 Hi-Z 缓冲区 36
5.3 光线步进 38
5.3.1 光线数据准备 38
5.3.2 生成采样方向 39
5.3.3 配置光线数据 40
5.3.4 Hi-Z 光线步进 41
5.3.5 处理碰撞信息 43
5.4 降噪处理 44
5.4.1 时域过滤 44
5.4.2 双边过滤 46
5.5 用户交互模块 47
5.6 本章小结 48
6 系统测试 49
6.1 测试环境 49
6.2 测试方案 49
6.3 功能性测试 50
6.4 健壮性测试 51
6.5 性能测试 52
6.6 本章小结 53

7 总结与展望 55
7.1 工作总结 55
7.2 工作展望 55
参考文献 57
致 谢 61

1 绪 论

1.1 课题研究背景

1.1.1 全局光照技术发展历程

全局光照技术[1]（Global Illumination，以下简称 GI）是真实感渲染中的重要一环，它本质上是对真实世界中的光照进行模拟，如图 1.1 所示。

图 1.1 全局光照场景

在 GI 技术中，我们主要关心的是直接光照和间接光照这两部分：直接光照是从光源出发的，直接照射到物体表面上产生的光照效果；间接光照考虑了在物体表面之间的光线弹射所产生的光照效果，即：

$$global\ illumination = direct\ illumination + indirect\ illumination$$

经过几十年的发展，全局光照现今已有多种实现方向，较为常见的包括光线追踪、路径追踪、光子映射、基于点的全局光照、球谐光照、环境光遮蔽和基于体素的全局光照等。从 1968-1997 全局光照技术就提出了包含了路径追踪与光线追踪等方法的早期概念。光线投射是光线追踪系列算法中的第一次尝试，其基本概念于 1968 年由 Arthur Appel 提出[2]。1979 年，Turner Whitted 在光线投射方法的基础上，考虑了光与物体表面之间的作用，即让光线在物体表面以反射，折射以及散射方式继续传播，直到与光源相交。光线追踪方法的核心思想是：从相机出发，向成像平面（近裁剪平面）上的像素发射光线，找到该光线和场景物体之间的交点，如果该点处的表面是粗糙表面，则计算光源直接照射该点产生的漫反射颜色；如果该点处是镜面或者折射表面，则继续向反射或折射方向发射另一条光线，直到光线达到设定的最大递归深度。在前人的研究基础上，Kajiya

于 1986 年进一步建立了渲染方程（Rendering Equation）[3]的理论，并使用它来解释因光能传输所产生的各种现象。这一方程描述了场景中光能传输达到稳定状态以后，物体表面某个点在某个方向上的辐射率（Radiance）与入射辐射亮度等的关系。Kajiya 也于 1986 年提出了路径追踪算法的理念，该方法是第一个无偏（Unbiased）的全局光照渲染方法。

1.1.2 混合渲染管线发展历程

光线追踪技术通过模拟光线的传输过程，从而能够较为精确的模拟全局光照，是实现全局光照算法的一种有效方法。但是由于传统的光线追踪算法如路径追踪（Path Tracing）等需要大量的计算资源，因此在离线渲染领域应用广泛，在实时渲染中较难实现。

但是随着软硬件的逐渐发展，英伟达于 2018 年推出了新一代基于图灵架构的新系列 RTX 显卡，该架构中包含了专门用于构建加速结构和光线求交的光线追踪单元（RT Core）。光线追踪因此也逐渐应用于实时渲染领域[4]，如《地铁：离去》、《赛博朋克：2077》等主流 3A 游戏均加入了对实时光线追踪技术的支持。

实时光线追踪的计算复杂度很高，且高度依赖于特定硬件实现，且现阶段的实时光线追踪还不能完全替代光栅化的过程，只是应用于环境光遮蔽、实时阴影、反射和次表面散射等画面细节中。主流的游戏与引擎厂商将光栅化和光线追踪技术组合在一起，在渲染管线的不同阶段灵活使用不同的技术，这被称为混合渲染管线[5]（Hybrid Rendering Pipeline）架构。本文中的方法与混合渲染管线的思想类似，在光栅化阶段依然使用延迟渲染技术，在 GI 阶段使用屏幕空间下的实时光线追踪来计算场景中的间接光照。

1.2 课题研究意义

软硬件的发展使得 3D 数字应用之间的画面竞争越来越激烈，如何快速构建高质量的 GI 成为了应用开发厂商的一大难题。传统的基于预计算的 GI 技术不仅需要大量的时间来进行烘焙，而且场景美术人员每次进行场景修改和光照调整之后，都需要重新进行烘焙，这一过程极大降低了应用开发和迭代效率，而基于实时光线追踪的动态 GI 能够为美术人员提供实时的预览效果，极大提升工作效率。

另一方面，实时光线追踪的主要性能瓶颈在于如何加速光线和场景求交的过程，常见的加速手段是对场景进行层次划分，如构建 BVH 树和场景体素化，而现阶段的实时光线追踪由于高度依赖硬件进行求交加速，因此只能运行在特定架构的硬件之上。但是大部分用户的硬件水平达不到实时光线追踪的硬件要求，根据 2022 年 3 月 Steam 显卡

硬件调查显示（如图 1.1 所示），在占比最多的前 10 款显卡中，仅有两款入门级光追显卡（RTX2060 与 RTX3060）。基于屏幕空间的光线追踪技术可以提供渲染耗时可接受的实时 GI，提升画面质量和动态光影效果。并且该方法是在软件层面进行光线追踪，不依赖特定硬件支持。

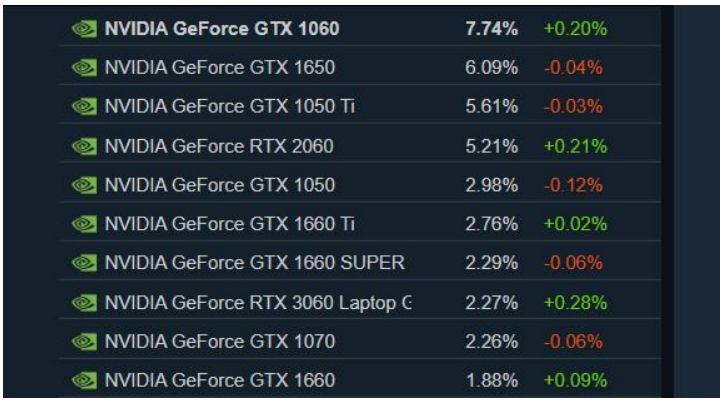

图 1.2 Steam 显卡硬件调查（2022.3）

此外，由于屏幕空间的光线追踪技术主要是在渲染管线中光栅化阶段结束之后进行的，和其他屏幕空间的算法类似，也可以视为后处理特效的一种。它只需要获取 G Buffer 和 Camera 中的一些纹理资源即可，不对光栅化、烘焙光照和其他后处理特效产生影响，因此和整个渲染管线是高度解耦的，可以快速应用于已有工程中，或者根据工程需要对方法进行自定义修改。

1.3 国内外研究现状

1.3.1 基于预计算的 GI 技术现状

游戏在运行过程中，除了渲染画面之外还有很多复杂的逻辑计算，而不少游戏为了追求画面质量，其性能瓶颈主要都在图形渲染部分上。为了减少渲染耗时，针对场景中的静态物体，游戏引擎通常会预计算其光照信息，并将其以某种数据结构存储起来，在游戏运行的时候实时采样即可，这样对于静态场景，可以获得较高的帧率和不错的光照质量。

如在 Unity 引擎中，场景美术人员通过在场景中布置一系列的光源，然后使用 Progressive Lightmapper（Unity 2018 版本之前使用 Enlighten，现已弃用）来生成场景中的光照贴图。在 Unreal 4 引擎中，则是使用 Lightmass 来生成场景中的全局静态光照。虽然静态光照不支持动态物体的光影变化，但是在移动端等低性能平台中依然有很广泛的应用场景。

根据存储载体的不同，基于预计算的全局光照技术主要分为基于烘焙贴图的全局光

照（Baked GI）和基于光照探针的全局光照[6]（Light Probe Based GI）。前者一般使用离线渲染中的路径追踪渲染器来模拟场景中的间接光照，将预计算的光照信息存储在光照贴图中，在游戏运行的时候直接读取光照贴图中的信息用于物体光照渲染即可。后者一般在场景中放置一系列稀疏分布的光照探针，这些探针可能是人工摆放或者是程序自动生成的，光照探针使用球谐函数[7]来对周围的光照信息进行编码。

1.3.2 基于屏幕空间的实时 GI 技术现状

渲染管线在完成了光栅化后，得到了帧缓冲的各种信息，但还未最终输出到屏幕上之前，我们可以在这一阶段中对上一阶段的渲染结果进行修改。屏幕空间中的几何信息有限，其算法复杂度一般和场景复杂度无关，因此基于屏幕空间的 GI 算法具有计算开销较小，运算速度快，易于应用等特点，国内外许多科研机构都对屏幕空间的 GI 技术做了研究。由于 GI 是对真实世界中的光照效果的近似模拟，这种模拟可以分为很多具体效果，而在屏幕空间中，被广泛应用的效果是屏幕空间环境光遮蔽[8]（Screen Space Ambient Occlusion，简称 SSAO）和屏幕空间反射[9]（Screen Space Reflection，简称 SSR）。

所谓环境光遮蔽（Ambient Occlusion，简称 AO），是指某个着色点被周围的几何表面所遮挡，从而降低了接受周围环境光的比例，因此整体光照效果会变得更暗，这种遮蔽现象常常发生在凹表面。AO 是一种十分常见且重要的光照效果，对于单个模型，可以通过预计算的方式计算其 AO 系数并存储在一张专门的纹理中，并在运行过程中对其进行采样。而对于整个场景的 AO 效果，可以在屏幕空间中进行近似模拟。

而对于另一种重要的光照现象反射，它通常发生在水面、玻璃等光滑表面上。而反射的模拟在屏幕空间也是易于实现的，它遵循这一基本假设：被反射的物体基本都已经在屏幕中被绘制过了。因此利用屏幕空间中已有的各种信息，我们可以高效地找到物体表面所反射的像素点位置，并计算反射效果。

常见的商业游戏引擎中都提供了后处理特效堆栈，其中就包括了对这两个特殊效果的模拟。

1.3.3 基于 3D 空间的实时 GI 技术现状

光线追踪可以很好的模拟场景中光线传播的过程，从而获得精确的 GI 效果，但是因其计算复杂度过高无法应用于实时渲染领域，但是可以通过模拟光线在 3D 空间中的有限次（通常是两次）传播来获得场景的间接光照信息，从而获得较高质量的实时 GI。

CryEngine3 引擎在 2013 年推出的游戏《孤岛危机 3》中，首次应用了光照体积传

播算法[10]（Light Propagation Volumes，简称 LPV）。LPV 算法效率较高且可以获得较高质量的间接光照效果，因此被很多商业引擎所采用，如 Unreal 4 引擎等。但是由于其采用了拟合出来的虚拟光源来进行间接光照，因此存在一定的漏光和不自然的现象，近两年主流引擎大多都弃用了该方法。

Nvidia 于 2011 年推出了体素全局光照算法[11]（Voxel Global Illumination，简称 VXGI），VXGI 通过对整个场景进行体素化，然后记录将光源注入每个小体素格子中来计算场景中的间接光照。该技术通过插件的方式集成到了 Unreal 4 引擎中，整体光照效果非常优秀，但是由于需要实时对场景进行体素化，且对于每个着色点都需要做锥形光线追踪，因此对硬件性能要求较高，Nvidia 在其 Maxwell 架构的系列显卡中对体素存储等进行了专门优化。

1.4 论文研究内容

本文基于 Unity 引擎内置渲染管线中的延迟渲染路径，设计并实现了一套屏幕空间下的实时光线追踪方法，从而实现高质量的实时 GI 效果。本方法首先实现基于延迟渲染管线和屏幕空间的光线步进技术来模拟光线的传播过程，并使用层次 Z 缓冲[12]（Hierarchical Z-Buffer）技术对光线步进进行加速优化。此时渲染的结果是充满噪点的，因此通过综合运用时域过滤和双边过滤对其进行降噪处理，将噪点控制在可接受的水平。

整个过程在渲染管线的后处理阶段（Post-Process）进行，由 C#脚本驱动，包括准备渲染数据，完成渲染设置，构建加速结构，发送渲染指令等操作。为了对方法进行加速，加速结构构建、光线步进和降噪处理等部分均在 GPU 端完成。

1.5 论文的组织结构

本论文结构安排如下：

第 1 章，绪论。介绍了课题的研究背景、研究意义、国内外研究现状、论文的研究内容、论文的组织结构。

第 2 章，相关技术。简要介绍了屏幕空间下光线追踪方法所需要的相关技术，包括渲染管线、渲染方程和蒙特卡洛积分等内容。

第 3 章，需求分析。对屏幕空间光线追踪方法进行分析，从实际工程角度出发进行功能需求与非功能性需求分析。

第 4 章，系统设计。针对整体方法和功能模块进行了详细的设计，具体包括了构建层次 Z 缓冲、光线步进、降噪处理、优化与输出、演示场景以及 UI 控制模块等子系

统的设计。

第 5 章，系统实现。基于 Unity 引擎内置渲染管线中的延迟渲染路径，对屏幕空间中的光线追踪方法进行了实现，并在演示场景中实现了 UI 控制模块，允许动态参数配置并实时预览效果。

第 6 章，系统测试。通过在 Sponza 场景中对 GI 效果进行实际演示，测试了各个功能模块的健壮性和稳定性，最后给出了不同渲染质量下的实际性能表现。

第 7 章，总结及展望。对整个研究工作进行了总结，并讨论了方法的优缺点以及对未来发展的相关期望等。

2 相关技术

本章节首先介绍了渲染管线中的不同阶段和坐标变换，并对屏幕空间光线追踪所依赖的延迟渲染路径进行了概述；然后对屏幕空间和 3D 空间两种不同类型的 GI 方法进行了简要介绍；最后介绍了本文方法的实现平台 Unity 引擎。

2.1 渲染管线

渲染管线（Render Pipeline）也称为渲染流水线，是实时渲染的核心组件，它通过接收从 CPU 端发送的各种渲染数据和绘制命令，从一个虚拟三维场景出发，经过一系列变换、处理等操作，最终渲染出一副呈现在屏幕上的二维图像。

2.1.1 渲染管线中的不同阶段

渲染管线有着众多子阶段，根据划分的功能粒度不同，也有着不同的阶段划分方法。一种划分方法是将渲染管线分成四个阶段[13]，包括应用阶段、几何处理阶段、光栅化阶段和像素处理阶段，如图 2.1 所示。在每个阶段内部通常也是一个流水线系统，包含着很多子阶段。渲染管线的一大特点就是每个阶段的输入数据都是上一阶段的输出数据。

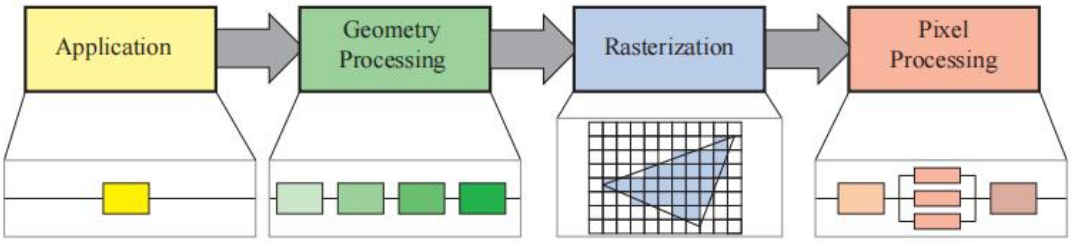

图 2.1 渲染管线的不同阶段[13]

渲染管线的第一个阶段——应用阶段主要是在 CPU 上执行的，该阶段的主要任务是将三维场景的各种数据，如：顶点数据、着色器代码、材质信息、灯光信息以及一些其他渲染设置发送给 GPU。这个由 CPU 端发起的，GPU 端执行的过程一般叫做 Draw Call，由于 CPU 和 GPU 之间的通信效率受到带宽等原因的影响，因此过高的 Draw Call 是影响程序运行效率的重要原因之一。

在几何阶段，GPU 接收来自 CPU 提交的渲染数据和绘制命令，该阶段的核心任务是通过一系列的空间变换将处于局部坐标系中的顶点，变换到屏幕坐标系中，在变换的过程中还会执行如顶点着色器[14]（Vertex Shader），裁剪等操作。在几何阶段结束之后，我们只获得了一系列顶点，还不能显示屏幕上的像素。

在光栅化阶段，会完成三角形设置和三角形遍历。光栅化是一个离散的过程，它会

根据三角形覆盖像素的情况，生成并组装片元。片元是一个状态集合，光栅化阶段利用三角形的重心坐标插值出三角形内部的片元属性，如法线方向、纹理坐标、屏幕坐标以及深度信息等。这一阶段通常还会进行一些抗锯齿操作，如多重采样抗锯齿[15]（MSAA）等。

在最后的像素处理阶段，会根据片元着色器（Fragment Shader），也叫做像素着色器[16]（Pixel Shader）对光栅化产生的片元进行光照计算以及阴影处理，最后通过可见性测试的片元会和颜色缓冲中的颜色进行混合。在最终输出到屏幕上之前，还会进行添加各种效果不同的后处理特效，屏幕空间的光线追踪方法便是在这一步发生作用，基于已经进行直接光照的渲染结果，利用光栅化阶段生成的信息如深度缓冲、法线缓冲等，来计算场景中的间接光照。

2.1.2 渲染管线中的空间变换

在渲染管线的几何阶段，会对三角形的顶点进行一系列的变换。渲染管线中所用到的所有几何操作几乎都可以在四维坐标空间中完成，它包含三个传统的几何坐标和第四个用于帮助处理透视观察的齐次坐标。这些 4D 坐标使用 4x4 的矩阵和 4 维向量来操作。

首先从单个模型出发，我们定义模型的平移、旋转和缩放三个变换矩阵，三个矩阵相乘就可以获得模型变换矩阵(Model Matrix)，模型变换矩阵与模型的局部坐标系（Local Space）下的坐标相乘便可以得到在世界坐标系（World Space）中的坐标。

然后以相机为参考，将其变换到世界坐标系原点，方向指向 z 轴负半轴的位置上，这个变换过程的矩阵叫做观察变换矩阵（View Matrix），所有的模型此时都会应用相同的观察变换矩阵，从而变换到观察坐标系（View Space）中。

到现在为止我们获得的坐标均是三维空间下的坐标，为了获取二维图像，我们需要使用投影变换矩阵（Projection Matrix）将三维坐标投影到二维平面上，即裁剪坐标系（Clip Space）。裁剪坐标系是一个二维坐标系，其 x 轴和 y 轴的范围均是[-1,1]。实际的输出窗口是以屏幕或者窗口的分辨率而定义的，因此使用屏幕映射，将裁剪坐标系进过平移和缩放变换到屏幕空间（Screen Space），并最终将图像渲染到屏幕上。

这里的屏幕空间指的是具体的屏幕坐标系，和本文题中的屏幕空间有一些概念上的不同，会在后文中详述。

2.1.3 延迟渲染路径

在延迟渲染[17]（Deferred Render）被提出之前，实时渲染中常用的渲染路径叫做前

向渲染（Forward Render），这是一种逐光源、逐像素的渲染方法，即对场景中的每个动态物体和动态光源，都会进行一次光照计算。这种方法的优点是易于实现，缺点是当场景中的物体和光源较多的时候，其计算复杂度会迅速膨胀，假设场景中存在 N 个物体和 M 个光源，其计算复杂度是 O(N*M)[18]。因此在实时渲染中，往往会限制逐像素的实时光源的数量，或者是根据重要性选择有限个光源参与光照计算。

延迟渲染在光栅化过程中会记录场景模型信息并将其存储在 G Buffer[19]中（如图 2.2 所示），并在 Light Pass 中统一进行逐光源的计算，由于屏幕上的像素总数通常是一定的，因此其光照计算量相对于前向渲染大大减少，因此场景中可以支持更多的动态光源。在前向渲染中，由于无法得知整体场景的遮挡关系，因此对于很多被遮挡的物体和像素也进行了光照计算，但是这部分内容其实并不会最终显示在屏幕上，浪费了大量的计算资源。而在延迟渲染中，通过将渲染信息记录到 G Buffer 中，对场景中需要着色的像素进行预先筛选，最终只对有效像素进行光照计算。

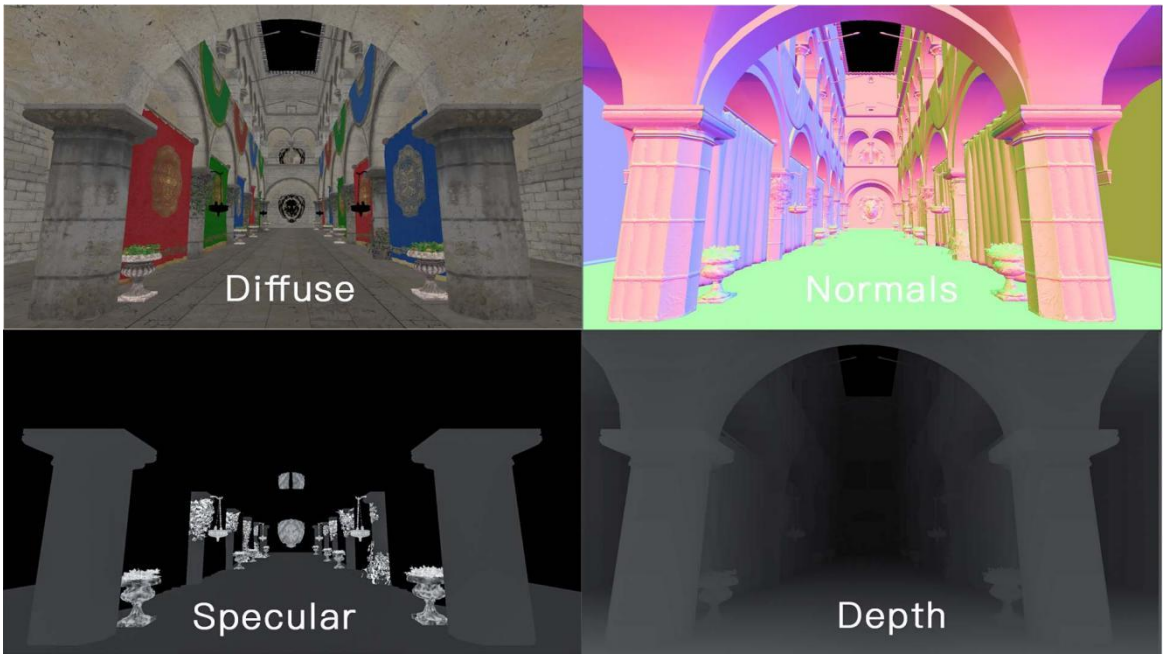

图 2.2 G Buffer 中的信息

延迟渲染的优势在于使用 G Buffer 剔除了大量的无效像素，但是需要每帧都渲染若干张 Render Texture，这对显卡的带宽提出了巨大的要求，因此在带宽敏感的设备中（如手机等移动设备）不太适用。其次，由于 G Buffer 中的每个像素只能存储对应模型的属性和信息，因此无法处理场景中的透明物体，且由于单个像素存储信息有限，所以基于硬件的抗锯齿（如 MSAA）无法用于延迟渲染[20]。另一方面由于使用 Light Pass 对所有的像素进行光照计算，因此场景中的模型只能使用相同的光照模型和材质。

基于屏幕空间的光线追踪方法，需要至少获得深度缓冲和法线缓冲这两个纹理，这可以在延迟渲染中的 G Buffer 中很方便的获取，不需要额外的开销，而在前向渲染中，则需要额外的开销来计算场景中的表面法线。

2.2 屏幕空间 GI 技术

这里提到的屏幕空间有三层含义，第一个是指相关算法大多都是在屏幕坐标系下进行的；第二个是指算法所使用前期数据全部来自于当前屏幕范围内的，而屏幕范围外的信息如场景几何信息、光照信息等是未知的；第三个指经过直接光照渲染呈现在屏幕上的渲染结果，这也是屏幕空间系算法的起点，即算法通过这张渲染结果，结合渲染过程中产生的有限的纹理信息等，来构建复杂的光照信息。

2.2.1 屏幕空间算法特点

在前向渲染路径中，我们在屏幕空间中可以直接获取的信息就是深度缓冲，它可以想象成一个包裹场景的壳，是对场景几何信息的一个简化描述，对于屏幕之外的场景信息将不被考虑。而在延迟渲染路径中，因为 G Buffer 的存在，我们可以屏幕空间中获取到更多的信息，因此算法也更加复杂多样。屏幕空间算法和延迟渲染的核心理念有异曲同工之妙，即算法复杂度和场景复杂度无关，对于屏幕外的信息一律不进行处理，因此屏幕空间中的很多算法都具有很高的运行效率[21]。

屏幕空间中的信息大多以纹理形式进行表达，如深度缓冲等，因此一个像素可以获取到相邻像素的信息，而在渲染管线中的光栅化和像素处理阶段，一个片元是很难得知相邻片元的信息的，因此在屏幕空间中可以完成如滤波和空间复用等在 3D 空间中难以实现的操作。

屏幕空间中进行的操作大多是后处理特效，它具有良好的兼容性，它不需要对渲染管线进行额外的修改和定制，即可实现各种各样的特效，这些特效对于提升画面质量是很重要的。

2.2.2 环境光遮蔽

SSAO 通过对着色点周围一定半径的球形范围区域进行像素采样，并根据深度缓冲，综合计算最终的遮挡系数用于环境光遮蔽，采样权重随采样点距离中心像素的距离而衰减。

水平基准环境光遮蔽[22]（Horizon-Based Ambient Occlusion，简称 HBAO）在着色点的四个随机方向上模拟光线步进，找到该方向上距离着色点一定范围内的最大水平角，

并使用额外的法线缓冲，一方面用于计算切面角，一方面用于剔除错误的遮挡关系。同时为了解决不连续问题，引入了距离衰减方程。

真实环境光遮蔽[23]（Ground-Truth Ambient Occlusion，简称 GTAO），从名字中可以看出其效果号称可以和离线渲染的结果媲美。GTAO 与 HBAO 类似，也对着色点的上半球范围内进行采样，但进一步进行了优化。首先是引入了 HBAO 中缺失的余弦项（cosine），考虑了不同采样角度的权重；同时考虑到环境光会在物体表面发生多次反射，将周围的物体照亮，因此我们得到的 AO 系数其实是偏小的，因此 GTAO 使用了路径追踪渲染器得到的 AO 系数作为参考，使用了拟合的多项式来对 AO 结果进行近似修正。

环境光遮蔽是一种廉价的 GI 效果，提升了画面的暗部细节和画面质感，且运行效率很高，因此在游戏中应用广泛；但是由于屏幕空间中的几何信息有限，无法得到视锥体之外的环境信息，因此环境光遮蔽的效果并不总是准确的。

2.2.3 屏幕空间方向性遮蔽

环境光遮蔽将间接光照考虑成一个常数，而屏幕空间方向性遮蔽[24]（Screen-Space Directional Occlusion，简称 SSDO）则更精确的考虑了间接光照。SSDO 在半球范围内随机采样，并根据深度缓冲，找到提供间接光照的采样点，将若干采样点提供的间接光照按权重进行叠加便是该着色点的间接光照。即 SSAO 增加的是明暗细节，而 SSDO 增加了周围物体对着色点的颜色影响，即俗称的 Color Bleeding 效果，如图 2.3 所示。

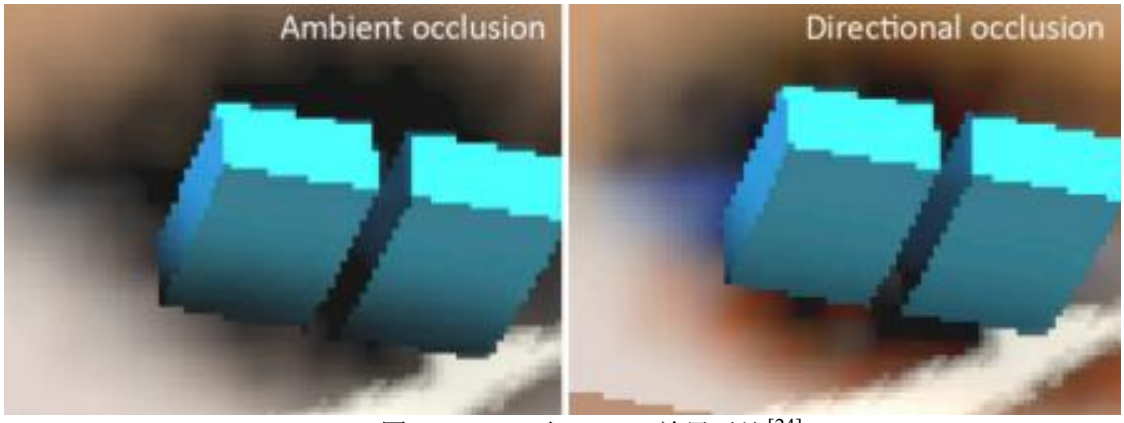

图 2.3 SSAO 与 SSDO 效果对比[24]

SSDO 使用了屏幕空间的几何信息，在遮挡关系上的判断有一定的不准确；且 SSDO 只适合小范围的间接光补全，它在实现中会限制一个有限的追踪范围，因此当距离超出设定好的追踪范围时，则无法补全。

2.2.4 屏幕空间反射

由于屏幕上被表面反射的物体有较大概率已经出现了屏幕上了，因此在屏幕空间来进行反射的模拟是十分高效的。屏幕空间反射（SSR）通过着色点的法线和相机视角位置，可以计算出光线的反射方向，对于光滑表面，可以从该方向投射一根光线，采用光线步进的方式每次前进一定步长，直到和其他表面相交，并根据碰撞信息采样像素颜色作为反射颜色即可。对于其他粗糙程度的表面，我们则对反射的锥形区域投射多条光线进行采样，最后对其进行降噪处理即可。

图 2.4 屏幕空间反射效果[25]

SSR 的具体效果如图 2.4 所示，它关注了场景中的高频反射信息，而本文中的方法则更加关注漫反射信息，其基本思路和 SSR 较为类似，但是在采样方法、具体优化上有着一些不同。

2.3 3D 空间 GI 技术

2.3.1 光照体积传播算法

光照体积传播算法（LPV）首先会根据 Shadow Map 找到接受直接光照的物体表面，对这些物体表面进行采样简化，最后获得一系列次级虚拟光源，如图 2.5 所示。然后 LPV 首先会对整个场景进行 3D 网格化，再将虚拟光源注入对应的网格中，再使用二阶球谐函数来存储各个网格的辐射度信息。在模拟光线传播的过程中，每个 3D 网格都可以向六个方向进行传播，并更新对应网格中的辐射度，经过几次迭代最终达到稳定状态。在最终渲染过程中，对于每个着色点，找到其所在的网格并计算所有方向的辐射度即可。LPV 算法效率较高且可以获得高质量的间接光照，但是存在一定的漏光现象。

LPV 技术在前十年的游戏中运用广泛，但是随着软硬件技术的进步，该技术已经被逐渐淘汰。

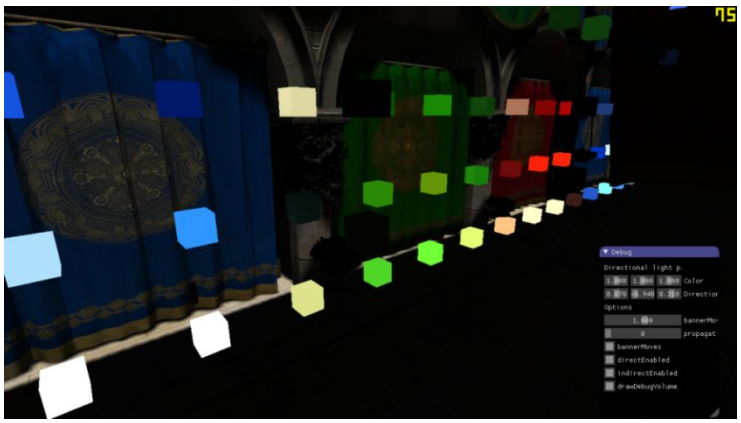

图 2.5 LPV 的虚拟光源[26]

2.3.2 体素全局光照算法

体素全局光照算法（VXGI）首先会对整个场景进行体素化，一般是八叉树结构（Octree）。然后找到接受直接光源的体素网格，并根据物体的表面法线和光源入射方向，通过表面的材质属性计算出光线出射方向的分布。和 LPV 从次级光源出发的方式不同，VXGI 从相机出发，对每个着色点做锥型光线追踪（或若干个锥形光线追踪，却决于表面材质），找到哪些次级光源可以照亮该着色点。VXGI 的全局光照质量非常好，和离线路径追踪的渲染结果很接近（如图 2.6 所示），但是由于对每个着色点都进行了锥形光线追踪，其计算开销相对于 LPV 大了不少，因此其应用也受到了一定限制。另一方面，VXGI 需要对场景进行体素化，在涉及动态物体的时候需要重新对场景进行体素化，也带来了额外的性能开销。

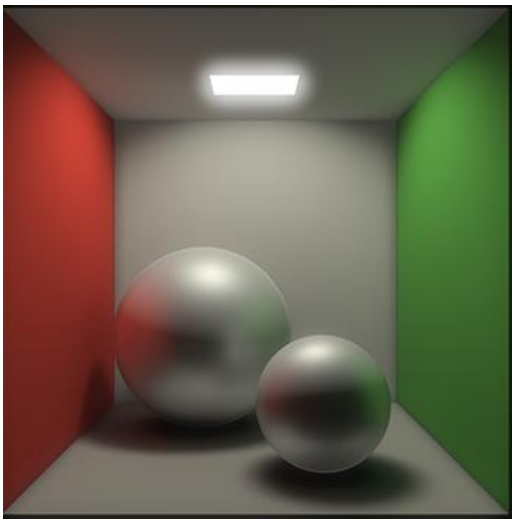

图 2.6 VXGI 的实时渲染结果[27]

2.4 Unity 引擎

Unity 是一款强大的跨平台引擎，可用于游戏、建筑、影视、动画、可视化等领域的 3D 数字应用制作。

Unity 引擎的渲染管线目前分为两部分，一部分是内置渲染管线（Built-in 管线），支持前向渲染和延迟渲染路径，开发者也可以通过 Command Buffer 对管线进行扩展。另一部分是可编程渲染管线[28]（Scriptable Render Pipeline，简称 SRP），它通过 SRP Core 将大量渲染功能封装成 C#代码，供开发者使用。Unity 官方使用 SRP 开发了两套渲染管线模板，即通用渲染管线（Universal Render Pipeline，URP）和高清渲染管线（High Definition Render Pipeline，HDRP），URP 针对通用开发平台，注重效率和通用性；HDRP 针对高性能主机和 PC，支持多种复杂光照。

2.5 本章小结

本章节详细介绍了渲染管线中的各个阶段核心功能、坐标变换以及延迟渲染路径。同时介绍了屏幕空间中的相关 GI 算法及其特点。最后介绍了本文方法的实现平台，基于内置渲染管线的延迟渲染路径实现。

3 需求分析

本章主要通过阐述屏幕空间下实时光线追踪方法的业务需求与应用场景，并从相关用户的实际开发需求出发，对该方法的设计与实现进行功能性需求和非功能性需求分析。在功能需求中提出了对该方法相关功能模块进行了分析；在非功能需求中对方法的性能、健壮性和易用性等进行了分析。

3.1 业务需求

近年来随着软硬件技术的不断发展，主流的游戏引擎和 3A 游戏厂商都开始了真实感渲染的军备竞赛，如 Unity 引擎针对次世代高性能平台推出的新一代高清渲染管线（HDRP），以及 Unreal 5 全新的实时 GI 技术 Lumen 等。但是在注重实时性和交互性的三维数字应用（如游戏、交互式展馆）中，渲染模块只是众多功能模块中的一部分，且很可能会成为最关键的性能瓶颈。因此为了保证渲染的实时性，主流厂商仍然采用光栅化为核心渲染方式，在传统光栅化渲染管线的基础上，在部分阶段中使用全新的光线追踪技术来获取更加真实的画面细节，从而提升整体的画面质量，即混合渲染管线。

由于当前的实时光线追踪技术大多需要依赖特定的硬件单元且计算复杂度很高，因此在实时渲染中，光线追踪技术只能用来实现提升特定画面细节的地方，如次表面散射、环境光遮蔽、反射等方面。而在提升画面细节方面，基于屏幕空间的相关算法由于实现简单，计算复杂度较低，且与传统核心渲染流程的兼容性较强，因此在现代游戏中被广泛应用。本方法的核心思路就是将光线追踪技术和屏幕空间算法的优势相结合，从直接光照的渲染结果出发，在屏幕空间中进行光线追踪，来高效、高质量的计算场景中的间接漫反射光照，并最终为场景提供高质量的 GI 效果。由于该方法类似于传统渲染流程中的后处理特效，因此可以快速应用于现有的项目中，提升场景画面的画面细节与真实感。且方法不依赖特定的硬件实现，可以针对设备性能提供不同等级质量的 GI 效果，因此可以在中低性能和不支持主流光线追踪硬件的设备上运行。

本方法由于需要获取光栅化渲染中生成的深度缓冲和法线缓冲等数据，而这些数据可以无消耗的从延迟渲染路径中的 G Buffer 中获取。由于 Unity 引擎的 HDRP 的目标平台是次世代高性能主机和 PC，不适合在中低性能平台上运行；且 URP 的延迟渲染路径刚推出不久，仍然处于不稳定的版本，因此本方法基于 Unity 引擎的内置渲染管线的延迟渲染路径实现。

3.2 用户需求

本方法基于主流游戏引擎 Unity 实现，使用内置渲染管线中的 Command Buffer 来对传统渲染流程进行一定修改并提交相应的渲染命令。在应用开发阶段，将运行在引擎的开发环境中，和引擎的其他功能组件以及性能分析工具相配合使用。本方法及功能模块的目标用户是针对中低性能平台开发、且对实时 GI 效果有一定需求的 Unity 应用开发团队，这类用户对于实时 GI 效果有以下三点需求：对于高质量实时 GI 效果的渲染需求、适用于不同性能平台且能自定义调整渲染质量的性能需求、与 Unity 渲染流程和其他开发工具的兼容性需求。

3.2.1 高质量实时 GI 效果

随着基于物理的渲染技术的不断普及与发展，很多基于高性能平台的实时渲染渲染效果已经非常接近现实世界。在这其中最重要的效果便是 GI 效果，GI 效果由两部分构成，分别是被光源直接照射产生的直接光照，和在物体之间发生弹射最终进入相机的间接光照。其中直接光照效果如方向光可以使用 Shadow Map 等技术找到被光源直射的表面，然后计算光照效果。对于间接光照效果，由于需要考虑光线在不同物体表面之间的弹射（一般是弹射两次），因此计算更加复杂。如果高效的实时计算间接光照是实现实时 GI 效果的重要内容，也是最核心的用户需求。

3.2.2 自定义渲染质量及性能需求

次世代 3A 游戏通过加入了光线追踪提升画面的真实感效果，但是带来的也是对性能的更高要求，这些效果往往依赖于特定的硬件实现，且无法在中低性能平台上流畅运行。为了这些平台上也实现高质量的实时 GI 效果，这要求在方法设计上尽可能的考虑降低对性能的消耗，且为开发者提供充足的自定义渲染质量配置，通过适当调整渲染参数来适应不同性能的平台，保证应用运行时的流畅性。

3.2.3 Unity 渲染流程及开发工具兼容性

大多数商业应用开发都依赖于成熟的开发工具（如 Unity 和 Unreal 引擎等），这些开发工具通常拥有丰富的功能组件系统和成熟的工具链。其中 Unity 引擎以强大的跨平台性能、丰富的插件资源和灵活的开发方式，在商业化的三维数字应用开发中占据了不可替代的作用。在应用开发阶段，引擎的大多数的功能都运行在 Unity 编辑器中，为了保证编辑器中的各个功能都可以正常运作，且在应用发布之后也可以运行良好，方法的设计和实现也需要保证对于原有渲染流程和开发工具的良好兼容性。

3.3 功能性需求

本文中的方法原理是在屏幕空间下，从直接光照的渲染结果出发，使用实时光线追踪来计算场景中的间接光照，从而实现高质量的实时 GI 效果。整个流程包括光线追踪、加速优化、降噪处理、其他优化等。

3.3.1 屏幕空间中的光线追踪

为了实现在中低性能平台上流畅运行且不依赖特定硬件实现的光线追踪方法，我们首先要考虑对场景的几何信息进行简化。传统的光线追踪是在 3D 场景中进行光线投射，然后计算其和场景中物体的相交点，这涉及到了对场景模型和三角形的遍历，计算非常耗时，因此在 3D 场景中进行三角形遍历求交的方式是不可取的。而屏幕空间中可利用的几何信息较为有限，只有以纹理形式描述的深度缓冲（如图 3.1），这可以看成对场景几何信息的有效简化，即一个从相机视角出发，覆盖整个可视场景的壳。而且更重要的是，在光栅化过程中记录的场景深度缓冲，其中代表的每个像素最后都会显示在平面上，即这些被筛选出来的场景信息是对场景信息的高度有效概况，可以在很大程度上来描述场景的整体形状和细节。

图 3.1 场景颜色与深度缓冲

由于屏幕空间中的场景几何信息是通过纹理形式给出的，因此我们可以将在 3D 空间进行的光线求交转换到 2D 平面上进行，这将大大简化光线与场景的求交运算。在具体的求交方法上，可以选择性能开销较小的光线步进（Ray Marching），即每次光线向前前进若干个像素，通过比较当前到达的深度和当前像素的深度来判断是否和场景相交。

3.3.2 光线追踪的加速优化

传统的光线步进基于线性步长，被称为线性光线步进（Linear Ray Marching），其求交的质量和性能在很大程度上取决于步长的选取（如图 3.2）：若步长太大，则可能直接跳过表面，造成错误求交导致走样；若步长太小，则需要很多次步进才能和表面相

交，虽然保证了求交的精度但是其速度较慢，因此需要一种动态步长选择的加速策略。

常用的加速策略有层次 Z 缓冲（Hierarchical Z Buffer，简称 Hi-Z 缓冲）技术，该技术通过生成不同分辨率下深度缓冲，动态选择下一次前进的步长，从而加速光线追踪的收敛速度。该方法通过对深度缓冲进行池化处理，生成不同分辨率下的深度缓冲。具体的方法是通过获取当前层级的 Hi-Z 缓冲中四个相邻深度的最小值（距离屏幕最近）作为下一层级的深度。因此每一个 Hi-Z 缓冲的分辨率都为上一层级的一半，而深度缓冲便是最低层级的 Hi-Z 缓冲，不同层级的 Hi-Z 缓冲可以看成在不同精度下对场景几何信息的描述。

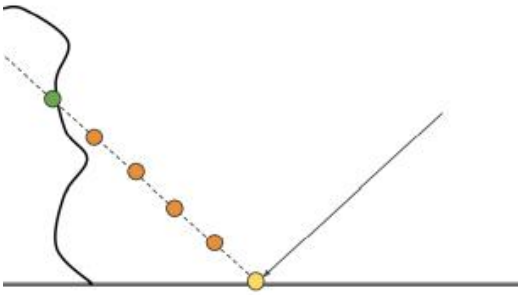

图 3.2 线性光线步进

因此在每一帧进行光线追踪之前，都需要先对光栅化生成的深度缓冲进行处理，进行加速结构的构建，生成一定层数的 Hi-Z 缓冲，然后在光线步进的过程中使用该缓冲进行求交加速。

3.3.3 光线追踪的降噪处理

由于计算资源的限制，我们只能对每个着色点发射有限根光线进行模拟追踪。根据目标平台的性能不同，通常是 8 到 16 根光线，这样获得的结果是没有充分收敛的，即使用光线追踪得到的结果会是充满噪点的，因此我们需要对生成的结果进行降噪处理。其实光线追踪能够在实时渲染领域得到应用，一个很重要的原因便是降噪技术的不断发展。

常用的降噪方法有时域过滤[29]（Temporal Filter）和双边过滤[32]（Bilateral Filter）。时域过滤是在多帧之间的降噪手段，它利用之前几帧的渲染结果，来和当前帧进行一定比例的混合，从而达到降噪的目的。双边过滤是一种非线性过滤，它可以达到保持边缘、降噪平滑的效果。双边过滤也是采用加权平均的方法，用周边像素亮度值的加权平均代表某个像素的强度，同时考虑了像素之间的其它因素作为权重，如深度、法线等。

3.3.4 其他优化

在得到场景光线追踪的结果之后，我们还需要对其进行一些额外的处理，首先是场景整体的亮度来调整整体的间接光照强度。其次由于是基于屏幕空间的方法，所能够利用的场景几何信息全部来源于屏幕上，即视锥体外部（不在屏幕内）的物体表面将不会对全局光照产生贡献和影响，因此在光源或者大面积的明亮表面离开视野的时候，会产生光照效果的变化。为了降低该变化造成的影响，这里使用基于像素 UV 的间接光权重贡献，对屏幕边缘进行渐隐处理。

另一个很重要的优化方法是在屏幕半分辨率下进行光线追踪，这样可以有效降低计算量，同时只损失一部分光照准确性。

3.3.5 用户交互模块

用户交互模块主要是对实现后的算法，在一些场景中进行测试和预览画面效果，其系统用例图如图 3.3 所示。

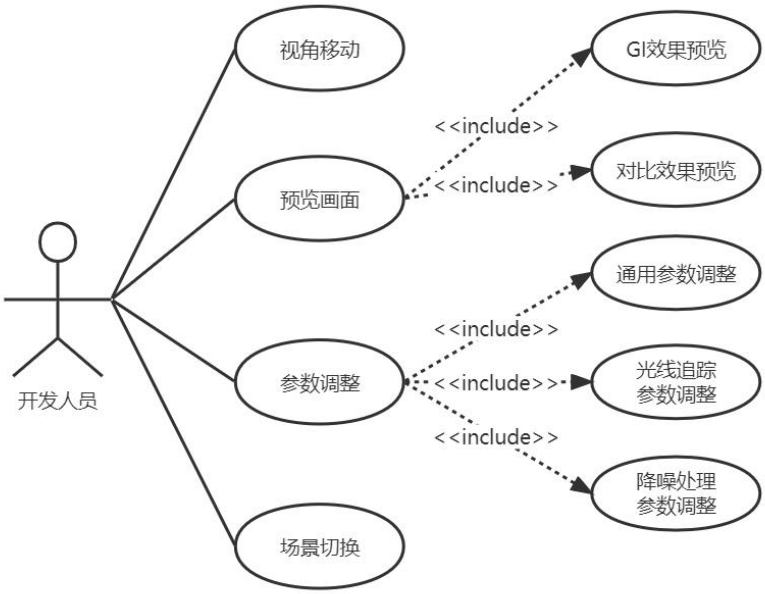

图 3.3 用户交互用例图

其中视角移动是用户通过键盘鼠标控制摄像机在场景中移动；预览画面可以实时在屏幕上预览当前的光照效果，同时可以预览对比效果；参数调整主要包括一系列控制渲染的参数设置；场景切换是用户通过键盘操作，在几个预设的测试场景中进行切换。

3.4 非功能性需求

3.4.1 性能需求

性能是实时渲染领域中非常敏感的问题，为了减少方法的整体渲染开销，因此要在

实现的过程中尽可能注重数据结构的高效利用，也要注意减少硬件之间数据传输，尤其是降低 CPU 和 GPU 之间的数据传输频率和带宽需求。在涉及大量坐标变换的过程中，也需要尽量减少不必要的性能开销。

3.4.2 健壮性需求

本方法的目标是在屏幕空间中计算场景的间接光照，并最终实现高质量的 GI 效果。GI 效果作为画面中的重要一环，它直接决定了整体的画面表现，因此我们需要 GI 效果尽可能的稳定可靠，在相同场景中尽可能表现出相同的质量。

同时作为引擎中开发中的一环，我们兼容处理不同种类的光源，不同种类的材质与着色器；同时也要和其他功能组件及后处理特效相兼容。

3.4.3 易用性需求

作为供美术人员使用的实时 GI 组件，我们需要保证其参数的简洁明确，合理设计参数的物理含义使其更加贴近于艺术家的审美感受，这样有利于对画面整体效果的调整，同时可以大幅提升美术人员的工作效率和迭代效率。

3.5 本章小结

本章主要通过对屏幕空间中的实时光线追踪方法的设计需求进行了相关分析，通过业务需求阐述了实时光线追踪和动态 GI 的市场需要；通过用户需求阐述了高质量的 GI 效果、可控的性能开销、自定义的参数配置和渲染流畅兼容性是相关开发用户最关心的内容；在功能需求中，通过阐述光线步进子系统，加速结构子系统，降噪子系统和其他相关优化等，分析了本方法在实现中需要考虑的功能内容；通过非功能需求描述了方法在设计和实现过程中需要充分考虑的其他设计细节。

4 系统设计

本章主要阐述本文方法的整体运行流程，以及相关功能模块的详细结构设计，且在光线追踪模块设计中给出了相关理论基础。基于屏幕空间的实时光线追踪所构建的动态 GI 技术以下简称 SSGI。

4.1 系统整体功能设计

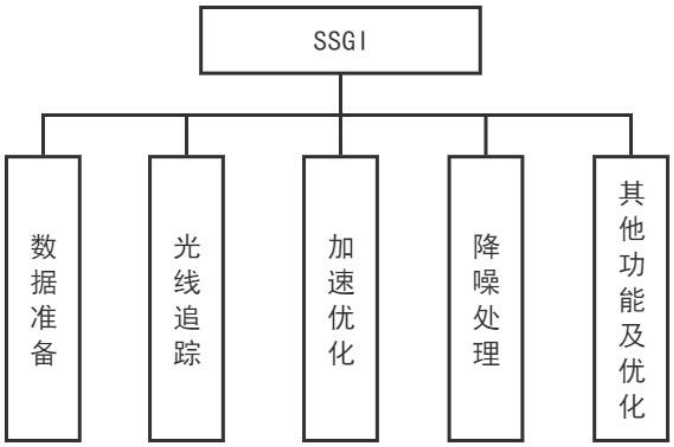

图 4.1 SSGI 整体功能模块

图 4.1 中展示了 SSGI 整体的功能模块设计，其组织架构和主流游戏引擎的常见光线追踪架构类似。数据准备是 SSGI 的准备工作，在这一步中，我们将在 C#脚本中初始化一系列资源，包括创建需要的 Render Texture、获取当前相机渲染生成的 Scene Color 等，以及设定后续渲染中的一系列参数，并在相机分辨率发生改变的时候对相关资源进行更新。光线追踪、加速优化和降噪处理是方法的核心部分。其中光线追踪负责一系列的坐标变换，配置光线数据并在场景中进行光线步进，最终得到和场景的碰撞信息，再对场景中的相关纹理进行采样，最终得到间接漫反射光照。加速优化主要包括加速结构的创建，并在光线步进的过程中应用加速结构。降噪处理是对充满噪点的光线步进结果进行时域过滤和双边过滤，将其噪点控制在可接受的水平。最后的其他功能及优化主要包括两部分，一部分是加入屏幕渐隐等优化处理，另一方面包括演示场景的渲染设置及 UI 控制模块等。

4.2 数据准备模块设计

数据准备模块是 SSGI 开始前的准备工作，这部分主要在 CPU 端完成。该模块主要包括两部分组成（如图 4.2），分别是纹理资源管理和参数管理。其中纹理资源主要是

在渲染过程中需要使用的相关 Render Texture，这按照功能用途可以将其分为以下四类：用于存储加速结构的纹理、用于保存光线追踪结果的纹理、用于降噪处理的纹理、用于获取当前相机渲染结果的纹理。渲染参数主要包括渲染流程中用于坐标变换的各种变换矩阵，还有用于控制渲染质量的各种参数，如投射的光线数量和光线步进的最大次数等。

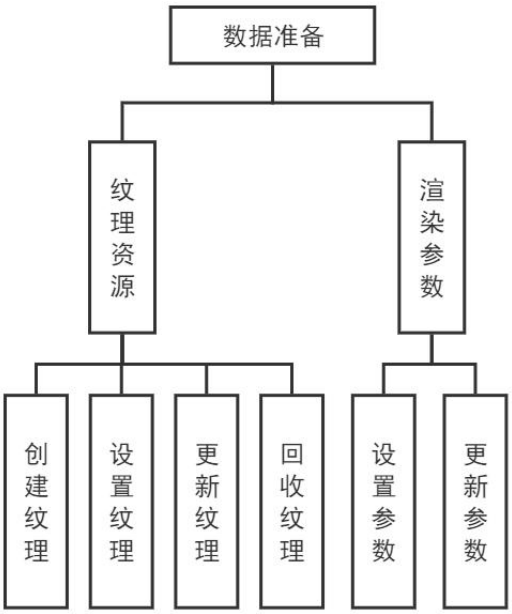

图 4.2 数据准备功能模块

对于纹理资源的管理，主要包括四个部分，首先是在 SSGI 初始化阶段，需要创建相应的纹理资源；然后通过 Command Buffer 对其进行设置，将其和 GPU 端的纹理资源进行绑定。由于这些纹理具有固定的分辨率尺寸，一般和相机分辨率保持一致，当相机分辨率发生改变的时候，我们也要同步更新这些纹理资源的尺寸，具体的方法是按照更新后的纹理尺寸重新创建这些纹理资源即可。当场景中关闭了 SSGI 效果，为了减少内存资源和带宽资源的消耗，因此我们需要将这些持有的纹理资源进行释放和回收。

和纹理资源类似，我们也要对渲染过程中需要使用的变换矩阵和相关参数进行设置与更新，这些参数可以直接通过材质进行设置绑定即可。其中这些变换矩阵大多和投影变换相关，即与场景中的相机设置有关，在具体的实现中，由于这些参数的整体数据量远不如纹理资源的数据量大，且为了动态调整渲染质量，因此这些参数一般实时更新，使得美术人员可以实时调整并预览画面效果。

对于纹理资源和矩阵等参数，都由 SSGIResource 类统一进行管理，它在类中持有这些资源，并含有相应的用于管理和更新的函数。对于自定义控制渲染质量的相关参数，在 SSGISetting 类中进行统一设置，该类继承自 ScriptableObject 类，这是 Unity 引擎中

一个用于持久化数据存储的类，通过继承该类，可以让开发人员手动创建配置文件，然后进行统一管理和初始化，是一种非常方便的数据配置方式。SSGIResource 在统一的管理类 SSGI 的调用下在适当的时机，创建资源，并根据相应的 SSGISetting 配置文件进行参数设置。SSGI 类继承自 MonoBehaviour，这是 Unity 生命周期中的重要基类，其中包含了一系列生命周期函数。图 4.3 是 SSGI、SSGIResource 与 SSGISetting 的类设计内容。

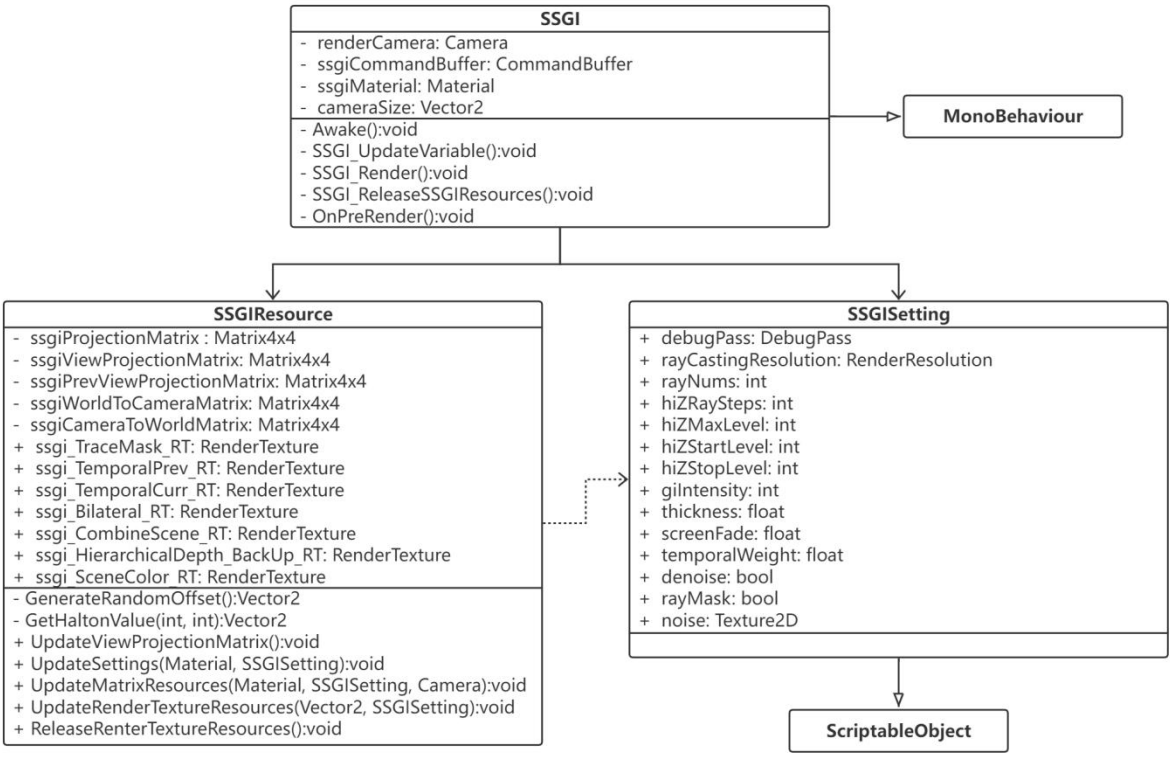

图 4.3 SSGI 类图

4.3 光线追踪模块设计

光线追踪是整个方法的最核心部分，通过蒙特卡洛积分方法来对渲染方程进行近似求解，来计算场景中的间接漫反射光照，光线追踪的部分是在屏幕空间中进行的，即所有的信息都来自于屏幕中。本小节首先对相关数学原理进行计算推导，然后对整个模块进行详细设计。

4.3.1 渲染方程与蒙特卡洛积分

场景中的光线从光源出发，在物体表面上相互作用与反弹，最终进入人的眼睛中。渲染方程在物理意义上给出了光线传输的这一过程描述，s 它是渲染领域中的核心概念，其数学描述如下：

$$L_r(x,\omega_r) = L_e(x,\omega_r) + \int_{\Omega^+} L_i(x,\omega_i) f_r(x,\omega_i,\omega_r)(n\cdot\omega_i)d\omega_i \qquad (4.1)$$

其中 L_r 是在位置 x 及立体角 ω_r 的出射光线；L_e 是在位置 x 及立体角 ω_r 的自发光项；L_i 是在位置 x 及立体角 ω_i 的入射光线。$f_r(x,\omega_i,\omega_r)$ 是双向反射分布函数（Bidirectional Reflective Distribution Function，以下简称 BRDF），该项决定了物体表面与光线的作用方式，描述了出射光线的分布，即物体的材质属性；$(n\cdot\omega_i)$ 代表了由入射角带来的入射光衰减。

根据线性算子理论（Linear Operator Theory），可以将方程（4.1）转换成一个线性系统的方程：

$$L = E + KL \tag{4.2}$$

其中的 E 是一个列向量，代表了材质的自发光项；L 是一个列向量，它包含了场景中所有的光源；K 是一个矩阵，代表了 BRDF 项。对方程（4.2）进行线性方程求解，得到：

$$L = (I - K)^{-1} E \tag{4.3}$$

对方程（4.3）中的矩阵进行泰勒展开可以得到：

$$L = \left(I + K + K^2 + K^3 + \cdots + K^n\right)\ E \tag{4.4}$$

其中 E 代表了场景中的光源和自发光；KE 代表了场景中表面的直接光照；K^2E 代表了场景中的间接光照（光线在物体表面上弹射一次）。这三项是近似模拟全局光照效果的重要组成部分。随着光线在场景中弹射次数的增加，K^nE 的贡献逐渐趋向于 0。

方程（4.1）由两项组成，分别是自发光项和上半球范围内单位立体角的入射光经过 BRDF 项作用后的积分项。该方程是一个高维的积分方程，我们无法求出其解析解，但是我们可以利用蒙特卡洛积分[34]来求取其近似解。这在全局光照技术中十分常见，因此任何全局光照技术的结果都是该方程的一个近似解。

蒙特卡洛积分形式如下：

$$\int_a^b f(x)dx = \int_a^b \frac{f(x)}{p(x)} p(x)dx = E\left(\frac{f(x)}{p(x)}\right) \approx \frac{1}{N}\sum_{i=1}^{N}\frac{f(X_i)}{p(X_i)} \tag{4.5}$$

其中 $f(X_i)$ 是被积分的函数，$p(X_i)$ 是随机变量 X 的概率密度分布函数。蒙特卡洛积分的核心思想是对于一个很难通过解析方式求解的定积分，我们可以通过对函数值进行多次随机采样求取均值作为对积分值的近似，且蒙特卡洛积分的近似结果是对积分值的一个无偏估计。

对于漫反射而言，其 BRDF 项为一个常数 k_α/π，其中 k_α 代表了漫反射系数。于是可以将方程（4.1）中的积分项改写成以下形式：

$$\int_{\Omega^+} L_i(x,\omega_i) f_r(x,\omega_i,\omega_r)(n\cdot\omega_i)d\omega_i \approx \frac{1}{N}\sum_{i=1}^{N}\frac{k_\alpha L_i(x,\omega_i)(n\cdot\omega_i)}{p(\omega_i)\pi} \tag{4.6}$$

在工程实现中，我们需要对着色点的上半球范围内进行均匀随机采样，并对采样结果求取均值。

4.3.2 光线追踪系统设计

该部分是逐像素进行的，即在每帧的渲染中，对于每个着色点都要进行上半球均匀采样和光线步进，且这个过程是高度并行化的，着色点之间的计算互不影响，非常适合在 GPU 端运行。光线追踪的结果是获得一张 Render Texture，用于后续的降噪和其他处理，因此光线追踪的部分主要是在片元着色器中实现的。

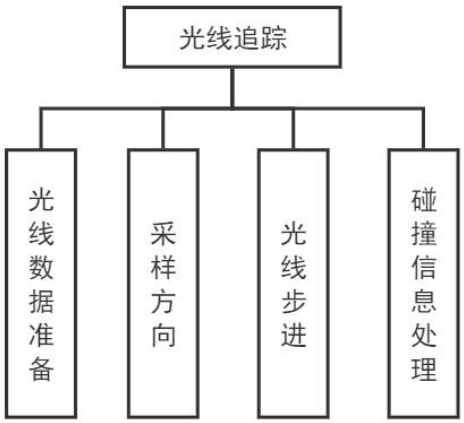

图 4.4 光线追踪功能模块

该部分主要分为四个子模块，如图 4.4，首先需要进行一系列的光线数据准备。首先需要对深度缓冲进行采样，得到屏幕空间中和观察空间的着色点坐标；然后对 G Buffer 中的法线缓冲进行采样，根据采样得到的世界空间法线，构建一组正交基底，用于对生成的采样方向进行坐标变换。

其次是生成采样方向，这一步通常使用一定的噪声图和随机序列来生成采样方向，但是不同的噪声具有不同的性质，随机的白噪声会给光线追踪的结果带来严重的噪点，即使对其进行模糊处理，但是仍然难以去除低频噪声。这里采用了 Inside 开发者在 GDC 2016 上的经验分享，使用蓝噪声来进行抖动采样[30]。蓝噪声和白噪声不同，它是一种高频的随机噪声，但是蓝噪声无法实时生成，因此可以使用一张预先生成的蓝噪声贴图（如图 4.5），在运行的时候进行抖动采样即可。为了采样蓝噪声图，我们需要额外生成一组分布均匀的随机序列，这里使用了 Halton 序列[31]。由于这里采样生成的方向是局部坐标系下的，因此我们需要应用世界空间中的正交基底将其转换后到世界空间中，然

后应用观察变换矩阵将其转换到观察空间中，在观察空间中计算得到光线方向的终点，然后再运用投影变换矩阵将其转换到裁剪空间中。

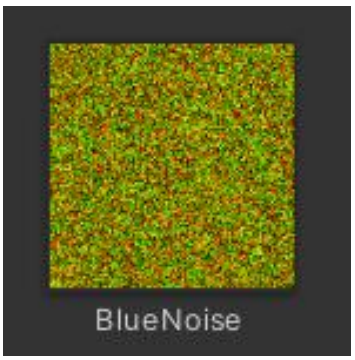

图 4.5 蓝噪声图

在光线步进中，我们根据已经计算好的光线数据（起点，方向等）进行光线步进，这是整个光线追踪模块中的性能瓶颈，其效率于质量直接决定了光线追踪的效率和质量。一般的线性光线步进每次步进一定的步长，通过当前步进的深度和像素的深度缓冲进行对比，来判断光线是否到达物体表面。为了整体性能的可控性，一般都会限制光线的最大步进次数，若光线的步进次数超出了最大步进次数，且仍没有和表面相交，这时会舍弃这条光线。常见的加速结构有层次 Z 缓冲技术，我们将在下一节对其进行讨论。

在判断光线是否和场景相交的时候，通过会作为一个范围限制，即当光线和场景壳距离在一定范围内时才认为二者相交。这里考虑到性能的开销，使用的是常量厚度，即认为场景中的所有表面都具有相同的厚度。更精确的做法是在光栅化过程中，剔除物体的正面，单独渲染背面来获得背面深度缓冲，再和深度缓冲相减来获得精确的物体厚度，但是会带来更高的性能消耗，这里使用常量厚度已经可以获得很好的效果了。

根据光线步进获取的碰撞结果，首先对交点的 Scene Color 进行采样，获取漫反射颜色，然后再对 G Buffer 中的深度缓冲进行采样，获得交点的表面法线，将交点的表面法线和当前着色点的表面法线进行点乘，来作为采样系数对之前获得的漫反射颜色进行修正。根据渲染配置的管线数量进行循环采样，获取最终叠加的漫反射颜色，在对其进行修正与求取平均值，最终将其输出。

4.4 加速优化模块设计

步长的选择直接决定了质量和性能，为了兼具效率和准确性，本方法使用 Hi-Z 缓冲技术来动态选择下一次前进的步长，这是一种试探回退的算法，它可以大大加速光线步进的速度。加速优化模块共有两个子模块，分别是构建加速结构和应用加速结构。如图 4.6 所示。

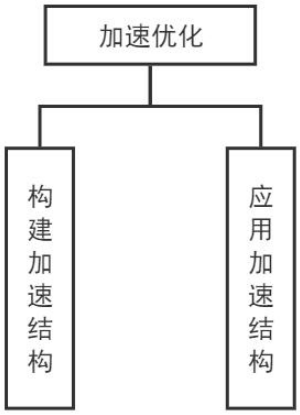

图 4.6 加速优化功能模块

在构建加速结构时，Hi-Z 通过对深度缓冲进行处理，选取周围四个像素深度的最小值，即距离屏幕最近的像素深度，作为下一个 Hi-Z 层级的像素深度。通常构建 5-7 层 Hi-Z 缓冲即可，过高的 Hi-Z 层级对于加速帮助十分有限，使用的频率很少。加速结构的构建应是完成数据准备之后进行的第一步，使用单独一个 Pass 来对其进行处理，使用 C#调用若干次渲染命令来绘制不同层级的深度缓冲，然后将其存储在一张 Render Texture 的不同 Mip Map 层级中。

在使用 Hi-Z 缓冲的光线追踪的阶段[33]中，首先会使用最小步长进行试探性步进，深度采样对应 level = 0 的 Hi-Z 缓冲区，并与深度缓冲进行对比。若没有相交则增大步长，深度采样对应更高层级的 Hi-Z 缓冲区，直到相交或者超出屏幕范围；光线与表面相交之后根据 Hi-Z 缓冲区向下级回退，直到得到相交的像素点坐标。其算法可视化[37]过程如图 4.7 所示。

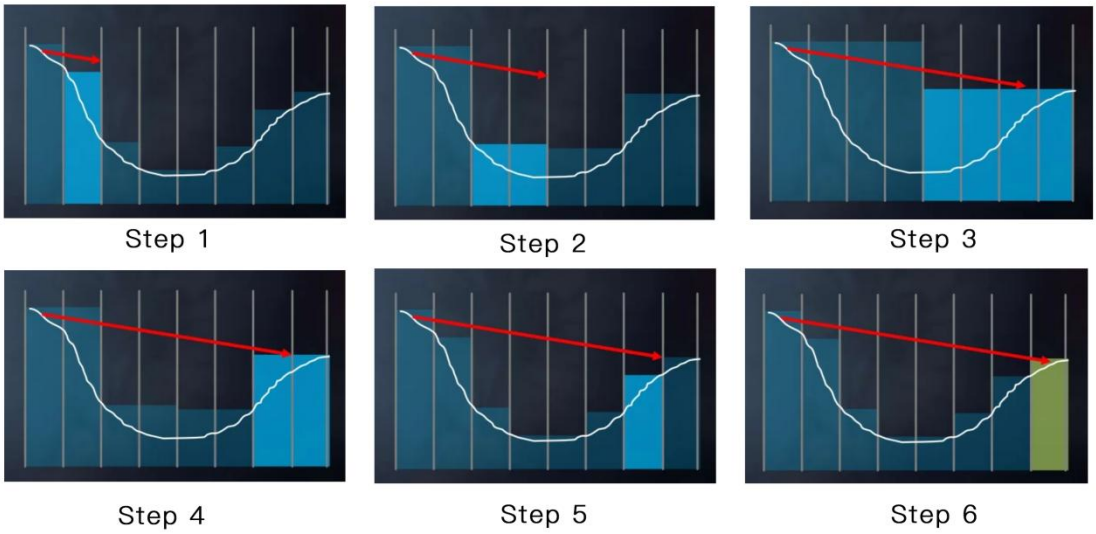

图 4.7 Hi-Z 算法步骤[25]

其伪代码如下：

```
void HiZTracing()
{
    mip = 0;
    while(level>-1)
        step through current cell;
        if(above Z plane) ++level;
        if(below Z plane) --level;
}
```

Hi-Z 的加速原理其实和层次包围盒（BVH）等场景划分技术的是类似的，Hi-Z 层级中的每个 Texel 相当于一个 AABB 包围盒，Hi-Z 层级越低，其包围盒越大。在具体的算法实现中，一般都使用包围盒是否相交叉来判断光线和场景的位置关系，其中具体由光线方向和光线与像素之间的深度关系决定：在光线方向为正方向（指向屏幕内部）且深度小于像素深度时（此时光线向着光线表面移动），应当增大 Hi-Z 的层级并继续迭代；在光线方向为正方向且深度大于像素深度时（此时光线和表面相交），应当降低 Hi-Z 的层级并继续迭代。当光线方向为负方向（指向屏幕外部）且深度大于像素深度时（此时光线向着光线表面移动），应当增大 Hi-Z 的层级并继续迭代；当光线方向为负方向且深度小于像素深度时（此时光线和表面相交），应当降低 Hi-Z 的层级并继续迭代。其中有两种情况在一次迭代过程中只需要降低 Hi-Z 层级即可，然后即可转换为另外两种情况。

4.5 降噪处理模块设计

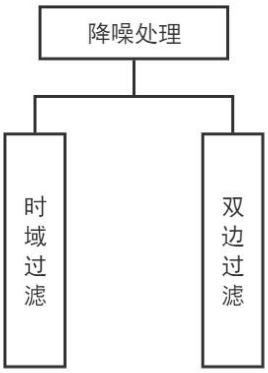

图 4.8 降噪处理功能模块

光线追踪的结果是充满噪点的，我们需要对其进行额外的降噪处理，由于是实时渲染，因此常用的降噪手段有基于多帧混合时域过滤和基于单帧权重的双边过滤两种，如图 4.8 所示。通过综合应用两种降噪手段，可以将噪点控制在可接受的水平。

4.5.1 时域过滤

时域过滤通过将之前几帧的信息和当前帧进行混合，来对当前帧进行降噪处理。本方法只对前一帧进行采样，首先我们需要得到当前帧和前一帧的区别，即 Motion Vector，在 Unity 中我们可以通过在着色器中声明相应的采样器即可获得[35]。但是我们不可以直接将对应位置的颜色值直接进行混合，因为之前的像素位置可能被遮挡，直接混合的化会导致发生错误。为了防止错误和拖影，当颜色值和前一帧差距过大的时候，就需要对前一帧的颜色进行舍弃。

这里我们是将采样范围限制在 3x3 的 9 个像素点中，通过权重获取每个像素点的颜色值，这里的权重选择参考了主流游戏引擎中的相关实现，使用的是优化过的 HDR 权重[36]。然后通过计算颜色的均值和标准差，并最终确定颜色的范围边界，然后对当前像素的颜色进行 Clamp，最后对颜色进行混合，这样可以保证当前帧和前一帧的色彩差距不会过大。

4.5.2 双边过滤

双边过滤是一种非线性过滤，它对渲染的当前帧进行处理，可以达到保持边缘、降噪平滑的效果。双边滤波也是采用加权平均的方法，双边滤波的权重不仅考虑了像素的距离关系，还考虑了像素范围域中的其他类型差异。在本方法中，使用了像素间的深度差异作为权重，将时域过滤的渲染结果再进行横向和纵向两次双边过滤，从而将噪点降到可接受的水平。这里采用了两次一维过滤来代替一次二维过滤，可以有效降低采样数量。

4.6 其他优化设计

4.6.1 屏幕边缘渐隐

由于是基于屏幕空间的算法，因此屏幕空间的算法缺陷是不可避免的，即场景外的光源和亮表面无法参与光照计算，为了减少突然的明暗变化，可以使用基于像素 UV 坐标的间接光照权重贡献，像素越接近屏幕边缘，则其权重贡献越少，整体看起来就是屏幕边缘的间接光照逐渐变弱。这一步可以在光线追踪的部分加入，在最后输出着色点颜色时加入基于屏幕 UV 坐标的修正即可。

4.6.2 半分辨率追踪

光线追踪的目标是生成屏幕间接光的颜色，其分辨率是独立于屏幕分辨率的，因此为了减少性能的消耗，可以在屏幕半分辨率的尺寸下进行光线追踪，这样可以增加每个

着色点的采样数量，整体效果也较为理想。要实现半分辨率下的光线追踪，在数据准备阶段就需要相应的修改 Render Texture 的分辨率，最后在混合输出的时候将分辨率整体提升到屏幕分辨率即可。

4.7 用户交互模块设计

在最终的方法演示 Demo 中，会将方法应用于具体的场景中，如 Sponza 场景和 Cornell Box 场景等，然后添加相应的 UI 参数控制模块，使得可以动态调整并实时预览画面质量。

4.7.1 功能设计

主要的交互控制及显示模块分为三个部分，分别是相机移动、UI 调整参数和实时参数（帧率、渲染耗时）显示，如图 4.9 所示。

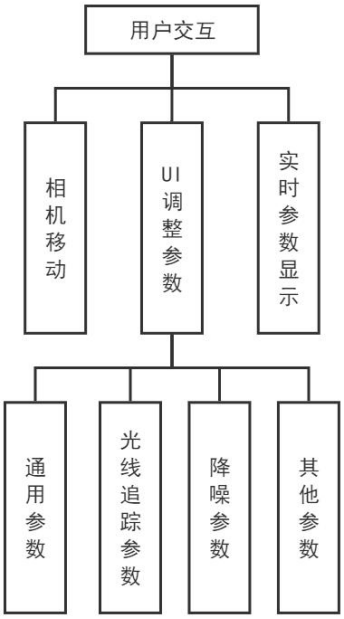

图 4.9 用户交互功能模块

对于相机移动，使用和 Unity 引擎 Scene 窗口类似的操作设计，即 WASD 建和鼠标进行操作，并可以使用 Shift 键进行移动加速。需要实时显示的参数主要有两个，分别是当前的帧率和当然的渲染耗时。在 UI 参数调整方面，按照功能模块划分成四类，分辨是通用参数：主要包括 GI 强度、光源方向、画面对比等；光线追踪参数：主要包括光线数量、步进次数、Hi-Z 缓冲区层数即起始终止层级；降噪参数：时域过滤、双边过滤；其他参数：屏幕间接光渐隐比例、场景壳厚度等。

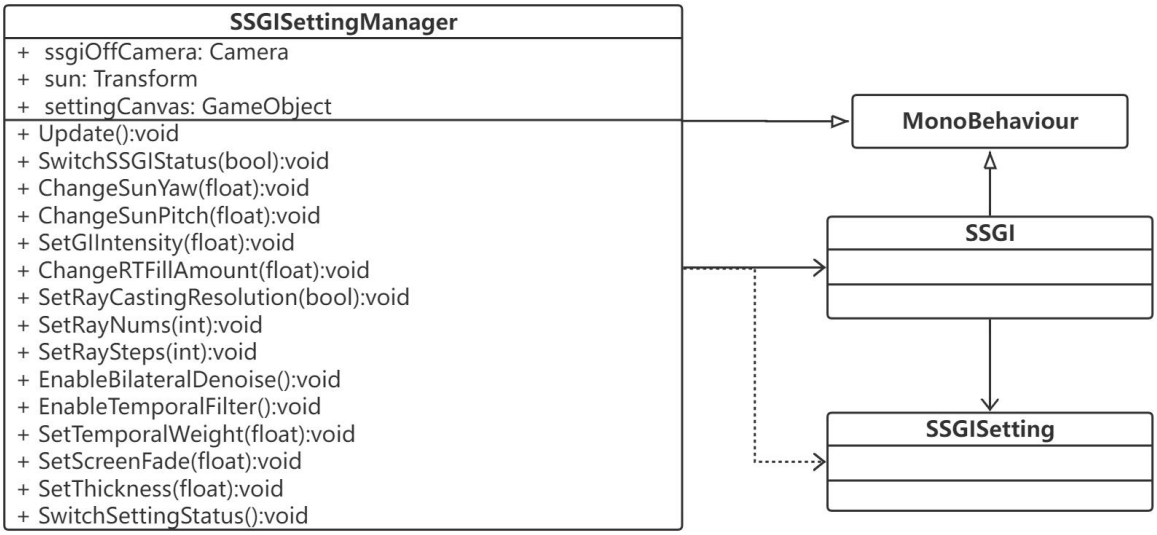

图 4.10 SSGISettingManager 类图

整体类设计如图 4.10 所示，使用 SSGISettingManager 来对场景中的 UI 进行动态管理，通过修改 SSGI 中 SSGISetting 来完成对渲染参数的控制。其中包含了 SSGI 效果的启用与关闭、与直接光照的对比效果和对具体参数的动态控制，提供了一系列用于按钮和滑块绑定的方法。

除了简单的 Cornell 场景，还会添加 Sponza 场景，它通常用来测试全局光照的效果。在整个场景中，只会添加一盏实时的方向光，除此之外场景中没有其他的光源，使用 SSGI 为其提供间接光照。除此之外还会添加使用烘焙光照的白天和夜晚的 Sponza 场景，然后再使用 SSGI 补充间接光照。

4.7.2 界面设计

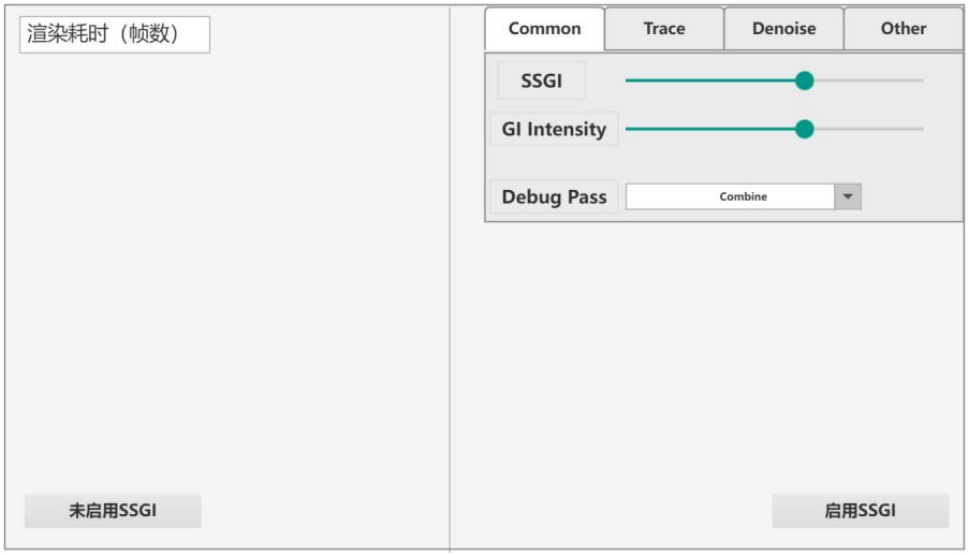

图 4.11 用户交互模块设计图

整个用户界面设计如图 4.11 所示，主要分为三个功能区，分别显示在屏幕左上角的实时渲染耗时和帧数，显示在屏幕左下角和右下角的信息提示和右上角的参数控制弹窗。在参数控制面板中有四个子面板，分别对应四类渲染参数，其中还会提供一个滑动条来实时控制 SSGI 画面在整个屏幕的占比，从而实时对比直接光照和全局光照的效果对比。在 Sponza 场景中，还会提供一组滑动条来控制场景中的光源方向。

4.8 本章小结

本章主要针对需求分析阶段提出的各项需求，从系统设计的角度进行了相应的设计实现。设计了数据准备模块来创建和绑定相应的纹理、矩阵和参数；设计光线追踪模块来在屏幕空间中进行光线步进，并给出了相应的理论基础；设计加速优化模块来构建加速结构并应用于光线步进；设计降噪处理模块对噪声结果进行降噪；设计对方法的其他优化；最后设计了用户交互模块来对方法进行演示和测试。

5 系统实现

本章将通过展示具体的工程实现来介绍 SSGI 的实现细节与各个步骤中的处理方法。整体流程如图 5.1 所示，包含数据准备、加速结构构建、光线步进、降噪处理和混合输出等部分，其中在章节 4.6 中所涉及的相关优化会体现在具体的实现细节中。在 Shader 中，构建 Hi-Z 缓冲、光线步进、时域过滤、混合输出分别对应一个 Pass，双边过滤对应两个 Pass。

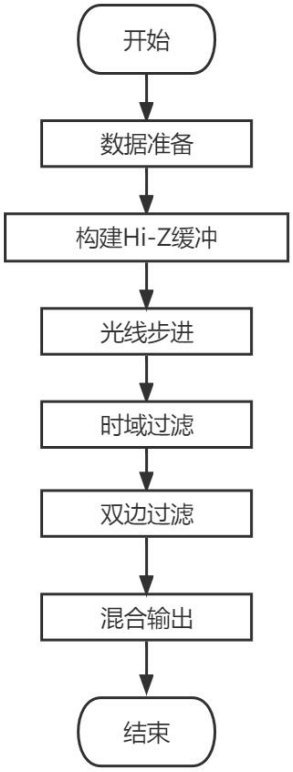

图 5.1 SSGI 整体流程

5.1 数据准备模块实现

在数据准备阶段中，我们主要会进行两部分的工作，分别是创建参数配置文件和使用 C#进行动态资源管理。

5.1.1 参数配置文件

SSGI 使用专门的 SSGISetting 类进行渲染参数管理，该类继承自 ScriptableObject 类，通过 CreateAssetMenu 特性来指定其生成的文件名字与路径。这样我们可以根据实际项目需要，创建多个不同参数的 SSGI 配置文件实例，从而适应不同性能的目标平台。

配置文件中包含多种参数，如图 5.2 所示，主要的渲染和控制参数分为四类。第一

类通用控制参数（Common），其中 GI 强度，用来控制 SSGI 整体的间接漫反射光的强度；Debug Pass 用来在开发阶段进行测试，其中包含显示混合输出结果（Combine）、显示层次 Z 缓冲结果（HiZ）和显示间接漫反射光照（Indirect Illumination）。第二类是管理光线追踪的参数（Trace），主要是用于控制光线步进的相关细节，其中 RayNums 用于控制投射的光线数量，一般以 8-16 较为合适；HiZRaySetps 用于控制光线尝试步进的次数，一般以 32-64 较为合适。第三类是降噪参数（Denoise），用于控制相关降噪功能的开启和控制细节。最后一类是其他的一些参数（Other），主要是一些额外的优化控制。这四类配置参数在也对应了用户交互模块中的四类参数。

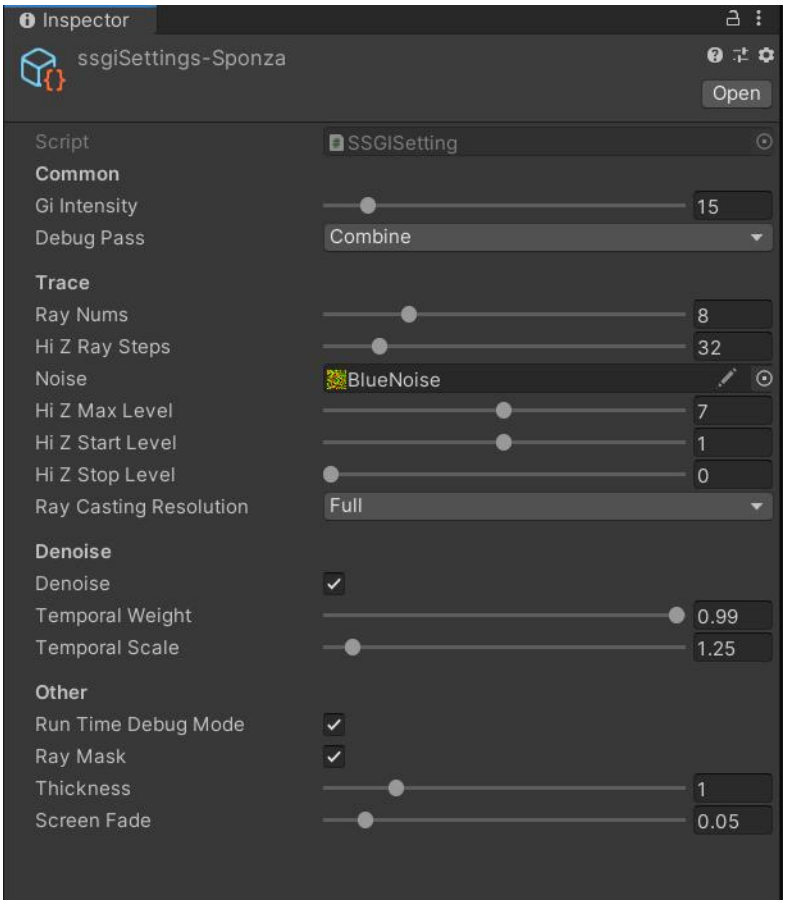

图 5.2 SSGISetting 配置文件

5.1.2 数据资源管理

数据资源管理主要由 SSGIResource 类实现，主要的数据资源包括各种渲染参数、Render Texture 和变换矩阵等，其中渲染参数由创建 SSGISetting 配置文件来指定。Render Texture 资源可以通过 Unity 引擎中的 RenderTexture 类来申请，并对其进行一些配置，其中用于存储 Hi-Z 缓冲的 Render Texture 需要开启 Mip Map。矩阵资源由于和相机设置相关，因此可以通过方法传入当前的相机来进行获取，并通过材质的 SetMatrix 方法来

进行设置。由于这些资源都是存储在内存中的，在渲染阶段需要传入 GPU 的显存，因此需要将这些资源和 Shader 中的相应资源进行绑定。由于光线步进的部分可以在屏幕半分辨率下进行，因此在申请 RenderTexture 的时候，也需要对其分辨率进行调整。

具体的 Render Texture 和矩阵申请、绑定的关键代码如下：

```
//decare RenderTexture ID and Matrix ID
public static int SSGi_HierarchicalDepth_ID=Shader.PropertyToID("_SSGi_HierarchicalDepth_RT"),
public static int SSGi_ProjectionMatrix_ID=Shader.PropertyToID("_SSGi_ProjectionMatrix"),

//Get and Set HierarchicalDepth RT
RenderTexture.ReleaseTemporary(ssgi_HierarchicalDepth_RT);
ssgi_HierarchicalDepth_RT = RenderTexture.GetTemporary(cameraSize.x,
                    cameraSize.y, 0, RenderTextureFormat.RHalf, RenderTextureReadWrite.Linear);
ssgi_HierarchicalDepth_RT.filterMode = FilterMode.Point;
ssgi_HierarchicalDepth_RT.useMipMap = true;
ssgi_HierarchicalDepth_RT.autoGenerateMips = true;
ssgiCommandBuffer.SetGlobalTexture(SSGi_HierarchicalDepth_ID, ssgi_HierarchicalDepth_RT);

//Get and Set Projection Matrix
ssgi_ProjectionMatrix = GL.GetGPUProjectionMatrix(renderCamera.projectionMatrix, false);
ssgiMaterial.SetMatrix(SSGi_ProjectionMatrix_ID, ssgi_ProjectionMatrix);
```

我们需要根据需求动态更新数据资源：其中矩阵资源和相机视角以及采样方向有关，因此需要每帧更新；当相机的分辨率发生变化的时候，纹理资源的分辨率也要相应的改变，和相机分辨率保持一致；渲染参数在应用实际发布之后应当保持不变，但是在开发调试阶段，我们可以选择让其每帧更新以实时预览画面效果，整体数据资源的更新逻辑流程图如图 5.3。

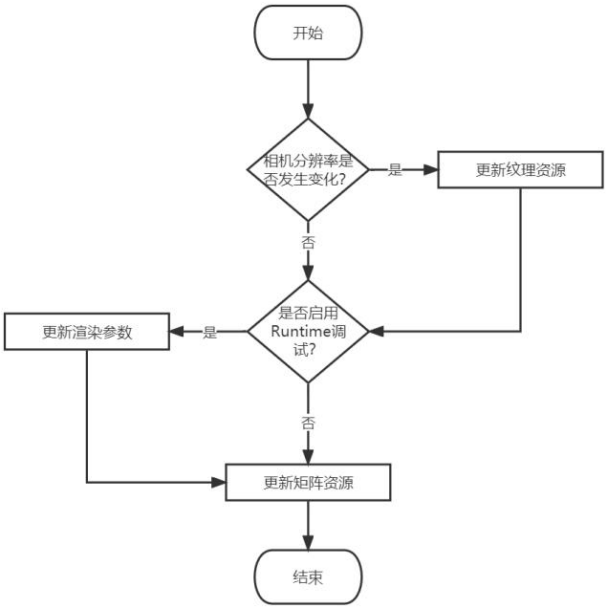

图 5.3 数据资源更新流程图

SSGIResource 类提供了相应的数据管理操作，在渲染流程中由 SSGI 类进行统一管理调用，该函数需要每帧调用，具体是在 MonoBehaviour 生命周期中的 OnPreRender 阶段执行，按照图 5.3 的流程图，具体实现代码如下：

```
private void SSGI_UpdateVariable()
{
    Vector2Int CurrentCameraSize = new Vector2Int(
        renderCamera.pixelWidth, renderCamera.pixelHeight);

    if (cameraSize != CurrentCameraSize)
    {
        cameraSize = CurrentCameraSize;

        //Update RenderTexture Resources
        ssgiResource.UpdateRenderTextureResources(cameraSize, ssgiSetting);
    }

    //Update Settings
    if (ssgiSetting.runTimeDebugMode)
    {
        ssgiResource.UpdateSettings(ssgiMaterial, ssgiSetting);
    }

    //Update Matrix Resources
    ssgiResource.UpdateMatrixResources(ssgiMaterial, ssgiSetting, renderCamera);
}
```

5.2 构建 Hi-Z 缓冲区

Hi-Z 缓冲区是用于加速光线步进的加速结构，它在数据准备阶段完成之后进行，由两部分组成。分别是运行在 CPU 上的 C#脚本，它负责发起渲染命令；和运行在 GPU 上的 Shader 代码，它负责具体的构建操作。首先将渲染过程中生成的深度缓冲赋值给一个用于构建 Hi-Z 缓冲区的临时 RenderTexture，然后设置当前的 Hi-Z 层级，并将对应的 Mip Map 层级作为当前的渲染对象，然后提交一次绘制命令，最后将渲染完成的 Hi-Z 缓冲区进行赋值即可，关键代码如下所示：

```
//Generate Hi-Z Buffer
ssgiCommandBuffer.Blit(
    BuiltinRenderTextureType.ResolvedDepth, ssgiResource.ssgi_HierarchicalDepth_RT);
for (int i = 1; i < ssgiSetting.hiZMaxLevel; ++i)
{
    ssgiCommandBuffer.SetGlobalInt(SSGIResource.SSGi_HiZ_PrevDepthLevel_ID, i - 1);
    ssgiCommandBuffer.SetRenderTarget(ssgiResource.ssgi_HierarchicalDepth_BackUp_RT, i);

    ssgiCommandBuffer.DrawMesh(GraphicsUtility.mesh,
        Matrix4x4.identity, ssgiMaterial, 0, RenderPass_HiZ_Depth);
    ssgiCommandBuffer.CopyTexture(ssgiResource.ssgi_HierarchicalDepth_BackUp_RT,
        0, i, ssgiResource.ssgi_HierarchicalDepth_RT, 0, i);
}
ssgiCommandBuffer.SetGlobalTexture(SSGIResource.SSGi_HierarchicalDepth_ID,
        ssgiResource.ssgi_HierarchicalDepth_RT);
```

在 Shader 部分，构建 Hi-Z 缓冲区是第一个 Pass，在顶点着色器（Vertex Shader）中，只需要传递像素的位置和 UV 坐标即可；在片元着色器（Fragment Shader）中会逐像素的进行操作，我们需要根据当前构建的 Hi-Z 缓冲区的层级，然后使用 SampleLevel 函数来对指定 Mip Map 层级的纹理进行采样，同时我们在最后一个参数中传入相对于该像素的偏移量，分别采样周围四个像素的深度值，最后取其中的最小深度（距离屏幕最近的深度）返回即可，相关代码如下所示：

```
float Hierarchical_ZBuffer(PixelInput i) : SV_Target {
    float2 uv = i.uv.xy;

    half4 minDepth = half4(
        _SSGi_HierarchicalDepth_RT.SampleLevel( sampler_SSGi_HierarchicalDepth_RT, uv,
            _SSGi_HiZ_PrevDepthLevel, int2(-1.0,-1.0) ).r,
        _SSGi_HierarchicalDepth_RT.SampleLevel( sampler_SSGi_HierarchicalDepth_RT, uv,
            _SSGi_HiZ_PrevDepthLevel, int2(-1.0, 1.0) ).r,
        _SSGi_HierarchicalDepth_RT.SampleLevel( sampler_SSGi_HierarchicalDepth_RT, uv,
            _SSGi_HiZ_PrevDepthLevel, int2(1.0, -1.0) ).r,
        _SSGi_HierarchicalDepth_RT.SampleLevel( sampler_SSGi_HierarchicalDepth_RT, uv,
            _SSGi_HiZ_PrevDepthLevel, int2(1.0, 1.0) ).r
    );

    #if defined(UNITY_REVERSED_Z)
    return max( max(minDepth.r, minDepth.g), max(minDepth.b, minDepth.a) );
    #else
    return min( min(minDepth.r, minDepth.g), min(minDepth.b, minDepth.a) );
    #endif
}
```

其中需要注意的是，在很有图形 API 早期，深度缓冲的值遵循近小远大的原则，如 DirectX 中近平面像素的深度为 0.0，远平面的像素深度为 1.0；而在 OpenGL 中，近平面像素的深度为-1.0，远平面的像素深度为 1.0。由于观察空间和裁剪空间的深度并非是线性映射，且由于 IEEE 754 规则下的浮点数大多分布在靠近 0 的位置，因此导致了高精度范围完全堆积在了近平面的位置，而远平面附近深度精度较低。所以现代往往会将深度缓冲反向，即近平面像素的深度为 1.0，远平面的像素深度为 0.0，从而使得各处的精度都得到了保证。在 Unity 引擎中，我们使用宏 UNITY_REVERSED_Z 来判断深度缓冲是否反向，若深度缓冲反向，则取像素深度的最大值；若深度缓冲未反向，则取像素深度的最小值。

在实际的实现中，一般生成 5-7 层 Hi-Z 缓冲即可，过高的 Hi-Z 缓冲层级对于整体效率并没有什么改善，此步生成的 Hi-Z 缓冲区如图 5.4 所示。

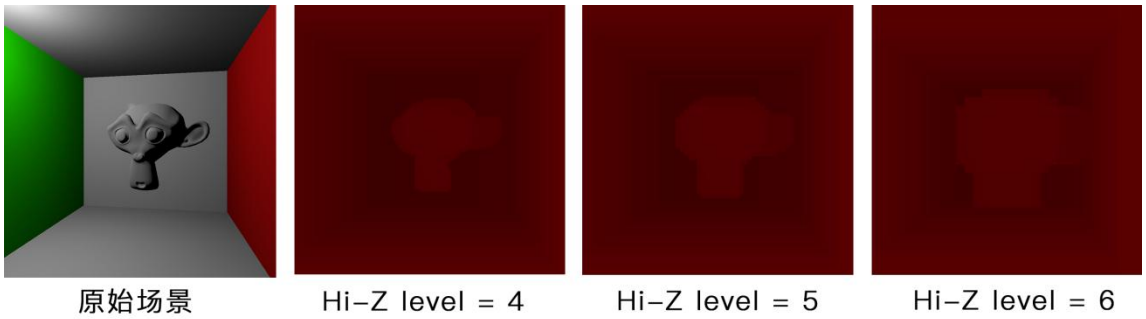

图 5.4 生成的 Hi-Z 缓冲区

5.3 光线步进

屏幕空间下的光线追踪的具体实现方式是光线步进，这是整个算法最核心的部分之一，该部分是运行在 GPU 上的，使用 Shaderlab 和 HLSL 代码进行编写。其中包括光线数据的准备与生成，使用 Hi-Z 缓冲区进行光线步进与碰撞信息的处理等内容，整体流程如图 5.5。

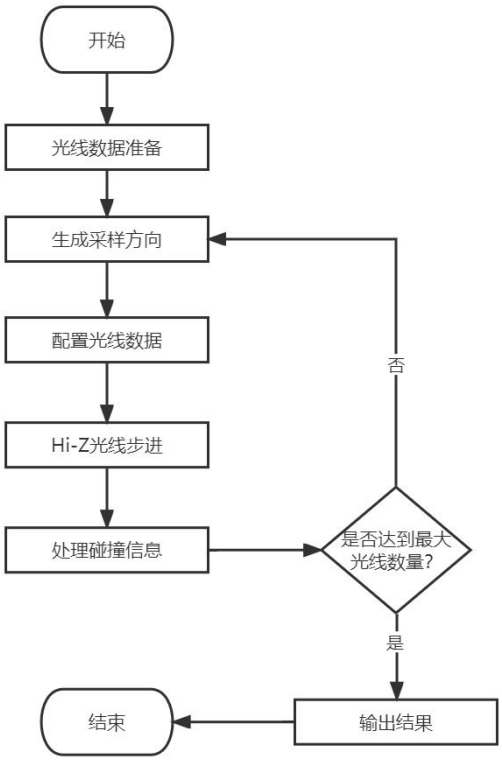

图 5.5 光线步进流程图

5.3.1 光线数据准备

在进行光线步进之前，我们首先需要通过一系列坐标变换生成光线等数据。在光线数据准备过程中，主要是计算三部分内容，分别是计算相关坐标、生成切线空间基底和

初始化数据。

首先是对深度缓冲进行采样，该采样的结果是裁剪坐标系下的深度数据，在章节 5.2 中我们提到这些深度数据的分布是非线性，因此我们需要使用 LinearEyeDepth 将深度数据转换到观察坐标系下的线性分布。同时使用裁剪坐标系下的深度数据通过坐标变换，获取屏幕空间和观察空间下的屏幕坐标，其中屏幕坐标应用平移变换即可，观察坐标则需要将屏幕坐标进行逆投影变换。

其次是对 G Buffer 中的法线缓冲进行采样，采样的结果是世界空间下的法线数据，这里需要以采样得到的世界法线为基准，构建切线空间下的基底，以供下一步对生成的采样方向进行处理，具体的生成方法是提供一个原法线方向差异明显的向量，然后利用两次向量叉乘生成两个和原法线垂直的向量即可。

最后是初始化输出数据，关键代码如下所示：

```
float2 UV = i.uv.xy;
//Get Depth Info
float sceneDepth = tex2Dlod(_CameraDepthTexture, float4(UV, 0, 0)).r;
float eyeDehpth = LinearEyeDepth(sceneDepth);
//Get Position Info
float3 screenPosition = float3(UV, sceneDepth);
float3 viewPosition = GetViewSpacePos(screenPosition, _SSGi_InverseProjectionMatrix);
//Generate Tangent Space Basis
float3 worldNormal = tex2D(_CameraGBufferTexture2, UV) * 2 - 1;
float3x3 tangentBasis = GetTangentBasis(worldNormal);
//Initialize Result Data
half outputMask = 0;
half3 outputColor = 0;
```

需要注意的是，对于一个着色点而言，这些数据是相同的，因此只要计算一次即可。

5.3.2 生成采样方向

使用均匀的随机序列或者白噪声图来生成的采样方向，在后续进行吗降噪处理之后，也难以完全消除生成的低频噪声。和白噪声不同，蓝噪声是一种高频的随机噪声，使用蓝噪声可以获得更好的渲染效果，由于蓝噪声难以实时生成，因此这里使用了一张预先生成的蓝噪声图，使用均匀随机序列对其进行采样即可。其中随机序列采用了 Halton 序列，每帧由 C#脚本生成，然后传入 Shader 中。生成 Halton 序列的关键代码如下所示：

```
private float GetHaltonValue(int index, int radix)
{
    float result = 0f;
    float fraction = 1f / radix;

    while (index > 0)
    {
        result += (index % radix) * fraction;
```

```
                index /= radix;
                fraction /= radix;
            }
            return result;
        }
        private Vector2 GenerateRandomOffset()
        {
            var offset = new Vector2(GetHaltonValue(m_SampleIndex & 1023, 2),
                GetHaltonValue(m_SampleIndex & 1023, 3));
            if (m_SampleIndex++ >= k_SampleCount)
            {
                m_SampleIndex = 0;
            }
            return offset;
        }
```

对蓝噪声的采样结果是一个二维向量，其 x 值和 y 值的范围都介于[0.0, 1.0]，即一个正方向区域，但是根据方程（2.1），我们需要对上半球范围进行采样，因此需要将方型采样范围转换为均匀的圆形采样范围[38]，相关关键代码如下所示：

```
float2 ToUnitDisk( float2 unitSquare )
{
    float2 p = 2 * unitSquare    - 1;
    float radius;
    float phi;
    if( abs( p.x ) > abs( p.y ) )
    {
        radius = p.x;
        phi = (PI/4) * (p.y / p.x);
    }
    else
    {
        radius = p.y;
        phi = (PI/2) - (PI/4) * (p.x / p.y);
    }
    return float2( radius * cos( phi ), radius * sin( phi ) );
}
```

5.3.3 配置光线数据

将章节 5.3.2 中生成的圆盘采样数据其作为半圆在平面上的投影坐标，根据球面方程计算出竖直方向上的坐标即可获得一个采样方向。需要注意的是，此时生成的采样方向是局部坐标下的，需要先利用章节 5.3.1 中生成的世界空间切线基底将其转换到世界空间中，再利用观察变换矩阵将其转换到观察空间中。

然后根据已经生成的数据配置光线数据，光线的起点即章节 5.3.1 中的 screenPosition；光线的方向我们需要将着色点的观察坐标沿采样方向平移获得方向终点，然后再将方向终点投影到裁剪空间下，再和光线起点相减即可获得。整体流程的代码如下所示：

```
//Sample blue noise texture
half2 hash = tex2Dlod(_SSGi_Noise, half4((UV + sin(i + _SSGi_Jitter.zw)) * _
    SSGi_RayCastSize.xy / _SSGi_NoiseSize.xy, 0, 0)).xy;
```

```
//Generate ray sample direction
float3 dir;
dir.xy = UniformSampleDiskConcentric(hash);
dir.z = sqrt(1 - dot(dir.xy, dir.xy));
float3 World_dir = mul(dir, tangentBasis);
float3 View_dir = mul((float3x3) (_SSGi_WorldToCameraMatrix), World_dir);
//Generate ray data
float3 rayStart = screenPosition;
float4 rayClipPosition = mul(_SSGi_ProjectionMatrix, float4(viewPosition + View_dir, 1.0));
float3 rayDirection = normalize((rayClipPosition.xyz / rayClipPosition.w) - screenPosition);
```

5.3.4 Hi-Z 光线步进

在这一步中，使用数据准备阶段生成的 Hi-Z 缓冲来对光线步进进行加速，这是整个方法中最消耗性能的一步，其主要流程如图 5.6 所示：

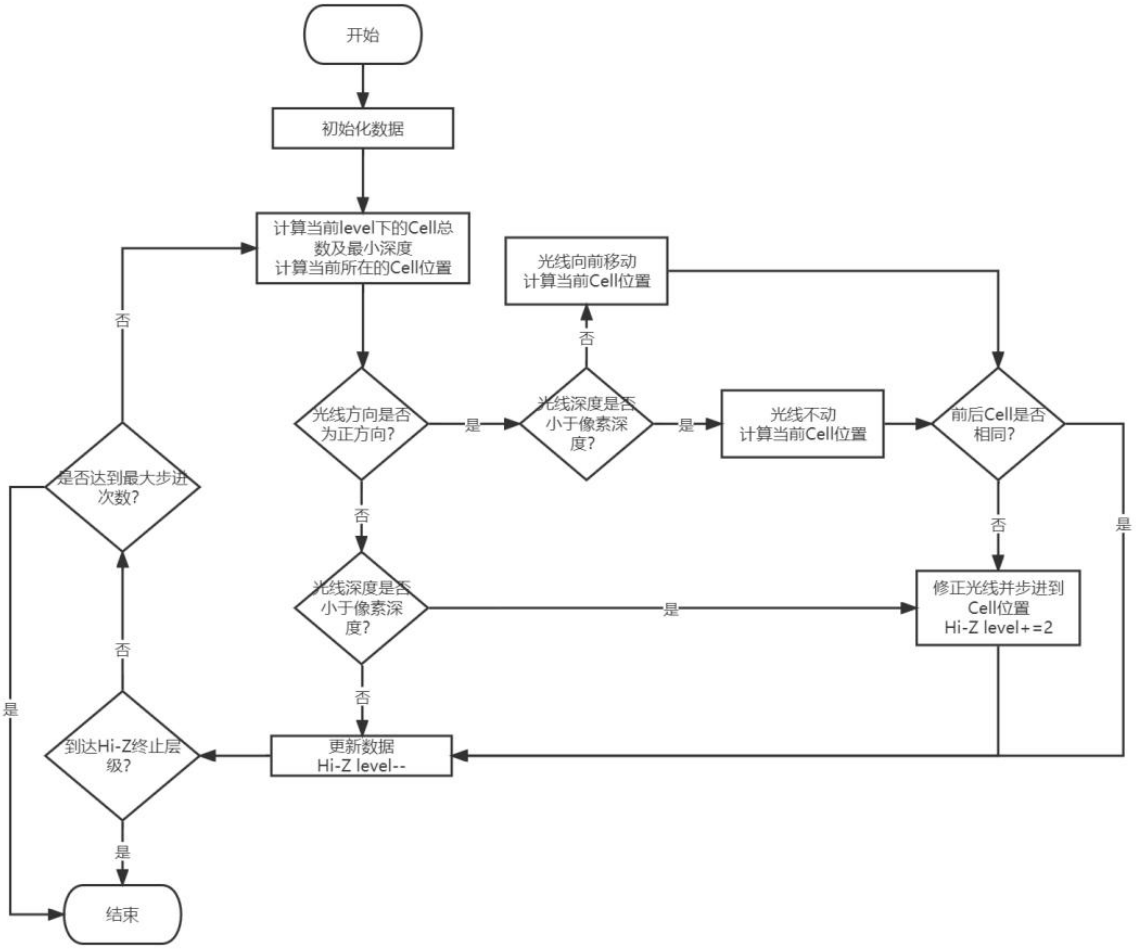

图 5.6 Hi-Z 光线步进流程

Hi-Z 缓冲区在不同分辨率中描述了场景的几何信息，在一个 Hi-Z 缓冲区中，相同深度的像素区域可以看成是一个 AABB 包围盒，我们也可以通过判断是否光线和像素是否处于同一个 AABB 包围盒来判断是否相交[40]。在进行 Hi-Z 追踪之前，我们仍然需要对已有数据进行一些处理，包括对初始光线位置进行修正、确定微小偏移等，相关代

码如下所示：

```
HiZ_Max_Level = clamp(HiZ_Max_Level, 0.0, 7.0);
rayOrigin = half3(rayOrigin.x, rayOrigin.y, -rayOrigin.z);
rayDir = half3(rayDir.x, rayDir.y, -rayDir.z);

float currentLevel = HiZ_Start_Level;
float3 ray = rayOrigin;

float2 cross_step = float2(rayDir.x >= 0.0 ? 1.0 : - 1.0, rayDir.y >= 0.0 ? 1.0 : - 1.0);
float2 cross_offset = cross_step * 0.00001;
cross_step = saturate(cross_step);

float2 hi_z_size = cell_count(currentLevel, screenSize);
float2 ray_cell = cell(ray.xy, hi_z_size.xy);
ray = intersect_cell_boundary(ray, rayDir, ray_cell, hi_z_size, cross_step, cross_offset);

int iterations = 0.0;
float mask = 1.0;
```

由于整体计算都是浮点类型，且可能会在屏幕半分辨率下进行光线步进，因此在切换 Hi-Z 缓冲层级的时候，光线此时的位置可能刚好在包围盒的边界，这会导致数值的误差，即随着步进迭代次数的增加，误差会越来越大。因此我们需要在步进的过程中对其进行修正，具体的做法就是当光线和一个新的包围盒相交的时候，对其位置进行修正，且进行微小的偏移，相关代码如下所示：

```
float3 intersect_cell_boundary(float3 rayOrigin, float3 rayDir, float2 cellIndex,
    float2 cellCount, float2 crossStep, float2 crossOffset)
{
    float2 cell_size = 1.0 / cellCount;
    float2 planes = cellIndex / cellCount + cell_size * crossStep;

    float2 solutions = (planes - rayOrigin) / rayDir.xy;
    float3 intersection_pos = rayOrigin + rayDir * min(solutions.x, solutions.y);

    intersection_pos.xy += (solutions.x < solutions.y) ? float2(crossOffset.x, 0.0) :
        float2(0.0, crossOffset.y);

    return intersection_pos;
}
```

接下来会进行光线步进的迭代，当到达指定停止迭代的 Hi-Z 层级或者达到最大步进次数时终止循环。在每次循环时，首先会计算当前所在的包围盒位置和当前当光线所在像素深度，具体的步进情况由光线方向和深度关系决定。在具体实现上参考了《GPU PRO 5》中的部分内容，具体的代码如下所示：

```
while(currentLevel >= HiZ_Stop_Level && iterations < NumSteps)
    {
        float3 tmp_ray = ray;
        float2 current_cell_count = cell_count(currentLevel, screenSize);
        float2 old_cell_id = cell(ray.xy, current_cell_count);
        float min_z = minimum_depth_plane(ray.xy, currentLevel,
```

```
            current_cell_count, SceneDepth);

        if (rayDir.z > 0)
        {
            float min_minus_ray = min_z - ray.z;
            tmp_ray = min_minus_ray > 0 ? ray + (rayDir / rayDir.z) * min_minus_ray : tmp_ray;
            float2 new_cell_id = cell(tmp_ray.xy, current_cell_count);

            if (crossed_cell_boundary(old_cell_id, new_cell_id))
            {
                tmp_ray = intersect_cell_boundary(ray, rayDir, old_cell_id,
                    current_cell_count, cross_step, cross_offset);
                currentLevel = min(HiZ_Max_Level, currentLevel + 2.0);
            }
        }
        else if (ray.z < min_z)
        {
            tmp_ray = intersect_cell_boundary(ray, rayDir, old_cell_id,
                current_cell_count, cross_step, cross_offset);
            currentLevel = min(HiZ_Max_Level, currentLevel + 2.0);
        }

        ray.xyz = tmp_ray.xyz;
        --currentLevel;
        ++iterations;

        mask = (-LinearEyeDepth(-min_z)) - (-LinearEyeDepth(-ray.z)) < Thickness
            && iterations > 0.0;
    }
```

最后以一个四维向量的形式返回碰撞信息，其中前三个分量存储的是光线与场景的交点信息，第四位存储的是一个遮罩，它通过将最小深度和此时光线的深度进行相减，再跟设定的厚度（Thickness）进行对比，来过滤掉一些错误或者无效的碰撞信息。

5.3.5 处理碰撞信息

根据获取的碰撞信息，对 Scene Color 和法线缓冲进行采样。其中对 Scene Color 的采样结果即该方向对着色点贡献的间接漫反射光照，但是需要根据颜色强度对其进行修正。对法线缓冲的采样可以得到碰撞点的世界法线信息，通过和采样方向进行点乘，从而获得其贡献的权重大小。返回的碰撞信息中的第四个分量是这该碰撞信息的遮罩信息，在这里会根据基于屏幕坐标 UV 的权重，来对间接漫反射颜色进行修正。

最后根据设定的光线数量全部采样完成之后，再进行均值化修正即可，关键代码如下所示：

```
    {
        ......

        //Sample Color and Normal
        float3 sampleColor = tex2Dlod(_SSGi_SceneColor_RT, half4(rayHitData.xy, 0, 0));
        float4 sampleNormal = tex2Dlod(_CameraGBufferTexture2,
```

```
            half4(rayHitData.xy, 0, 0)) * 2 - 1;
        float occlusion = 1 - saturate(dot(World_dir, sampleNormal));

        //Modify
        sampleColor *= occlusion;
        sampleColor *= rcp(1 + Luminance(sampleColor));

        outputColor += sampleColor;
        outputMask += Square(rayHitData.a * GetScreenFadeBord(rayHitData.xy,
                _SSGi_ScreenFade));
    }
    outputColor /= _SSGi_NumRays;
    outputColor *= rcp(1 - Luminance(outputColor));
    outputMask /= _SSGi_NumRays;

    //Output
    [branch]
    if (_SSGi_MaskRay == 1)
    {
        return half4(outputColor * saturate(outputMask * 2), eyeDehpth);
    }
    else
    {
        return half4(outputColor, eyeDehpth);
    }
```

此步最终输出的结果如图 5.7 所示，可以从第二幅图中看到，由于场景中右边的墙壁颜色为绿色，因此在左侧的墙壁和立方体表面都反射出了绿色；同理，由于场景中左边的墙壁颜色为红色，因此在地面上和相邻的立方体表面上都反射出了红色。第三幅图是叠加之后的结果，整体效果良好，但是由于此时还没有进行降噪处理，因此噪点较多。

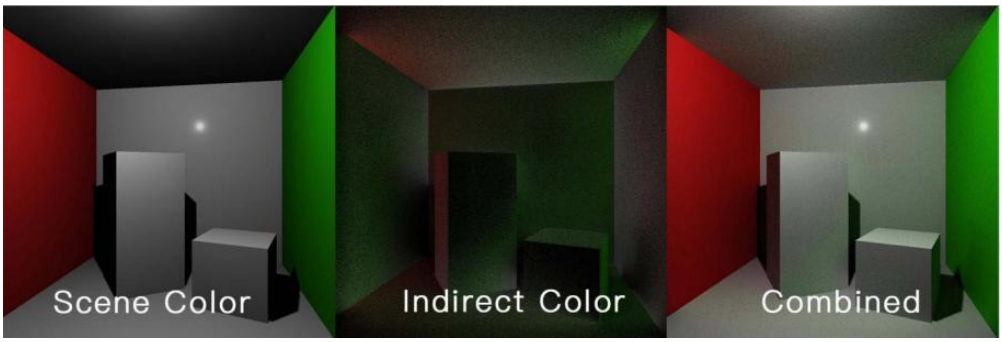

图 5.7 Hi-Z 光线追踪结果（rayNums = 8，Steps = 32）

5.4 降噪处理

在降噪部分，主要应用了时域过滤和双边过滤，在实际实现中，先应用了时域过滤，在应用双边过滤，其中双边过滤分别在 X 轴方向和 Y 轴方向进行了两次处理。

5.4.1 时域过滤

时域过滤的整体流程如图 5.8 所示：

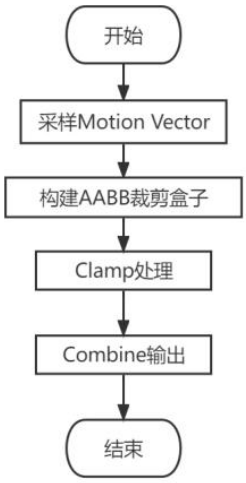

图 5.8 时域过滤流程图

时域过滤首先会采样 Motion Vector，获得当前帧和上一帧的差异向量。为了避免错误混合，需要对当前像素构建 AABB 裁剪盒子，计算颜色的具体范围，并根据颜色范围的上下界对前一帧的像素颜色进行 Clamp 操作，最后根据时域过滤的权重对当前像素颜色和上一帧的像素颜色进行插值输出，相关代码具体如下：

```
half4 Temporalfilter(Input i) : SV_Target
{
    half2 UV = i.uv.xy;
    //Get Motion Vector
    half2 Velocity = tex2D(_CameraMotionVectorsTexture, UV);

    //Get AABB ClipBox
    half SS_Indirect_Variance = 0;
    half4 SS_Indirect_CurrColor = 0;
    half4 SS_Indirect_MinColor, SS_Indirect_MaxColor;
    GenerateAABB(_SSGi_RayCastRT, 0, 10, _SSGi_TemporalScale, UV, _SSGi_ScreenSize.xy,
        SS_Indirect_Variance, SS_Indirect_MinColor, SS_Indirect_MaxColor, SS_Indirect_CurrColor);

    //Clamp Preview Color
    half4 SS_Indirect_PrevColor = tex2D(_SSGi_TemporalPrev_RT, UV - Velocity);
    SS_Indirect_PrevColor = clamp(SS_Indirect_PrevColor, SS_Indirect_MinColor,
        SS_Indirect_MaxColor);

    //Combine Preview and Current Color
    half Temporal_BlendWeight = saturate(_SSGi_TemporalWeight * (1 - length(Velocity) * 2));
    half4 SS_IndirectColor = lerp(SS_Indirect_CurrColor, SS_Indirect_PrevColor,
        Temporal_BlendWeight);

    return SS_IndirectColor;
}
```

在构建 AABB 裁剪盒子来计算像素颜色范围的时候，需要对周围 3x3 范围内的像素进行采样，这里参考了主流引擎的使用了优化过的 HDR 权重，使用范围内的像素均值和标准差来确定颜色的变化范围[39]，具体代码如下：

```
half4 m1 = 0.0; half4 m2 = 0.0;
for(uint x = 0; x < 9; x++)
{
    m1 += SampleColors[x];
    m2 += SampleColors[x] * SampleColors[x];
}

half4 mean = m1 / 9.0;
half4 stddev = sqrt( (m2 / 9.0) - pow2(mean) );

minColor = mean - AABBScale * stddev;
maxColor = mean + AABBScale * stddev;

filterColor = SampleColors[4];
minColor = min(minColor , filterColor );
maxColor = max(maxColor , filterColor );
```

5.4.2 双边过滤

为了减少计算量，因此将二维双边过滤展开成两次过滤，即分别在 X 轴方向和 Y 轴方向进行一次，分为两个 Pass 执行。这里采用了像素之间的深度差异作为双边过滤的权重，具体的权重计算代码如下所示：

```
float CrossBilateralWeight(float blurRadius, float r, float Depth, float originDepth)
{
    const float blurSigma = BLUR_blurRadius* 0.5;
    const float blurFalloff = 1.0 / (2.0 * blurSigma * blurSigma );

    float dz = (originDepth - Depth) * _ProjectionParams.z * 0.25;
    return exp2(-r * r * blurFalloff - dz * dz);
}
```

经降噪处理的结果如图 5.9 所示，可以看到综合应用时域过滤和双边过滤之后的结果，其噪点水平已经降低到了可接受的范围。

图 5.9 降噪处理结果（rayNums = 4，Steps = 32）

5.5 用户交互模块

整体界面使用 Unity 引擎的 UGUI 系统实现，主要分为信息提示和参数控制两部分，整体界面如图 5.10 所示。其中信息提示中的实时渲染耗时和帧数，是通过在 Update() 函数中使用计时器计算获得的。

参数控制是一个可弹出收起的弹窗，使用 Tab 按键或者点击相应按钮进行控制。参数控制面板有四个子面板，分别控制了不同类型的参数。其中在通用控制（Common）中提供了两个滑动条来控制场景中方向光的俯仰角（Pitch）和偏航角（Yaw）。提供了一个滑动条来控制画面对比的比例，通过在原相机相同的位置上放置一个辅助相机，该相机将渲染对象设置成和屏幕分辨率大小相同的 Render Texture，然后在 UI 层级进行显示，并利用图片遮罩来控制该 Render Texture 显示的比例。这样做会带来一定的额外渲染压力，因此当遮罩比例为 0（完全不显示）时，需要停止该辅助相机的渲染。

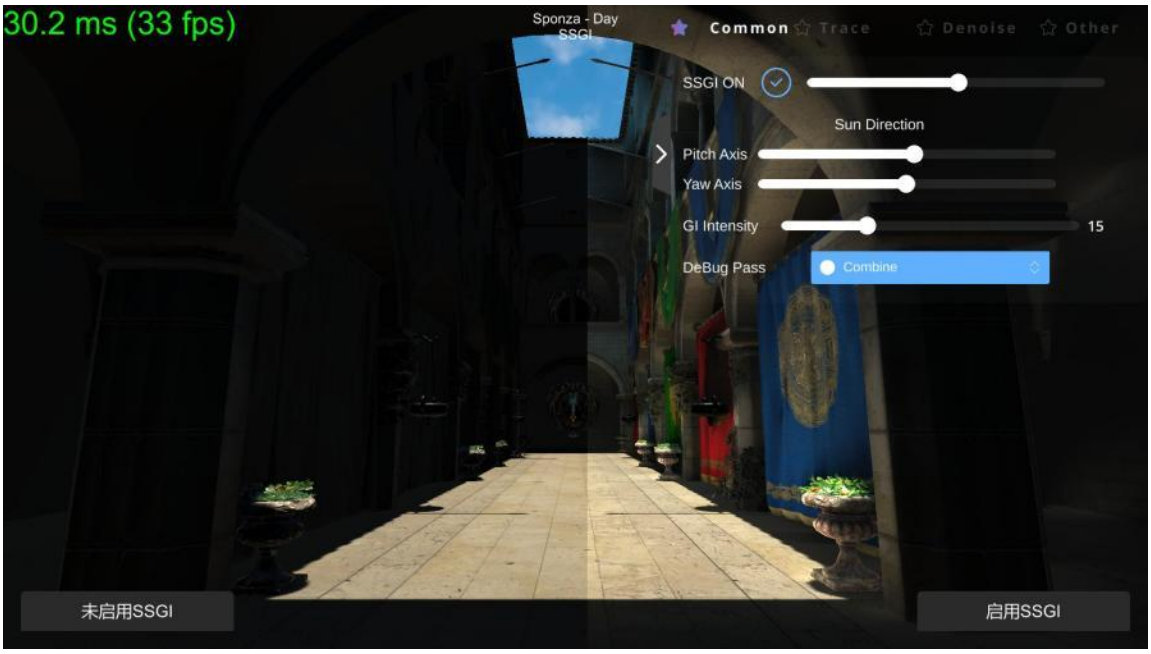

图 5.10 Sponza 场景和用户交互界面

相机控制参考了 Unity SRP 示例场景中的实现，使用 WASD 按键来控制相机前后左右的运动，使用 QE 按键来控制相机的上下运动，使用 Shift 按键来进行加速移动，按住鼠标右键的时候可以旋转相机视角。

系统中一共有四个演示场景，分别是直接光照的 Sponza 场景，有白天烘焙光照的 Sponza 场景，有夜晚烘焙光照的 Sponza 场景和 Cornell Box，由于 Unity 内置渲染管线中并没有实时的面光源，因此使用了实时的点光源进行测试。四个场景的相互切换分别对应数字按键 1-4。

5.6 本章小结

本章通过描述实现细节、给出关键代码和展示相关截图，详细介绍了 SSGI 的四个主要环节：数据准备、加速结构构建、光线步进、降噪处理，并在第五节中展示了 Sponza 渲染结果对比和用户交互界面。

6 系统测试

系统测试是验证整个系统可靠性的关键工作，是证明系统稳定性的重要因素。本章节将通过功能性测和健壮性测试等，对整个系统进行测试。

6.1 测试环境

系统整体测试环境如表 6.1 所示，其中显卡（GPU）为移动版的 1070，这是 2016 年发布的一款帕斯卡架构的显卡，具有 2048 个 CUDA 核心。该显卡在如今只能在 1080p 分辨率下以较低画质运行主流 3A 游戏，属于中等偏低性能的显卡；且该显卡没有专门的硬件光追单元，无法使用硬件光线追踪和 DLSS 等功能。

表 6.1 系统测试环境

表序号	测试环境	具体描述
1	操作系统	Windows 10 家庭中文版 1904
2	计算机型号	MSI GE63 VR 7RF
3	CPU	Intel(R) Core(TM) i7-7700HQ @2.8GHz
4	GPU	NVIDIA GeForce GTX 1070 Laptop
5	GPU 驱动版本	GAME READY 471.41
6	RAM	16G

6.2 测试方案

本系统的测试方案如下：对各个系统中的各个功能模块和子功能模块进行单独和组合功能测试，验证其功能可靠性；对系统中的边界条件与极端条件进行健壮性测试。

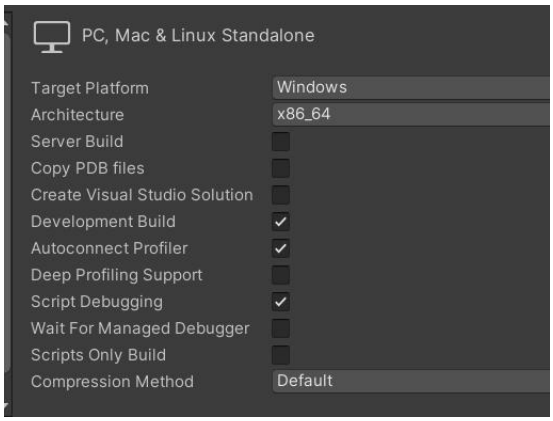

图 6.1 Build Setting 设置

由于 Unity 编辑器会带来额外的性能消耗，以及为了验证 Shader 编译及链接的正确性，因此所有的测试均在打包发布后的可执行文件上进行测试；在打包时可以在 Build Settings 中勾选 Development Build，并打开 Autoconnect Profiler 和 Script Debugging，这

样可以测试中通过 Unity 编辑器中的性能分析器 Profiler 查看实际渲染耗时，并检查程序功能是否有逻辑错误等。

6.3 功能性测试

功能模块主要分为五类，分别是数据准备、加速结构、光线步进、降噪处理和用户交互模块，每个功能模块下还有一系列子模块，针对这些功能的测试方法、预期结果以及测试结果如图 6.2 所示。

表 6.2 系统功能性测试用例

序号	测试内容	测试方法	预期结果	测试结果
1	创建数据资源	将创建出的纹理资源输出到屏幕上，矩阵资源打印日志	纹理显示正常、矩阵数值正确	正常
2	绑定数据资源	在着色器中应用矩阵变换，采样纹理资源并输出	纹理显示正常、矩阵变换正确	正常
3	更新数据资源	运行时修改相机分辨率	分辨率修改正常，纹理同步修改分辨率	正常
4	构建 Hi-Z 缓冲区	将 Debug Pass 调整为 Hi-Z	屏幕上显示场景的 Hi-Z 缓冲区	正常
5	Hi-Z 缓冲区层级数	调整生成的 Hi-Z 的最大层级	屏幕上显示修改过后的的 Hi-Z 缓冲区	正常
6	Hi-Z 缓冲区起始层级	调整 Hi-Z 缓冲区的起始层级	屏幕上的 Hi-Z 追踪结果发生相应改变	正常
7	Hi-Z 缓冲区起终止层级	调整 Hi-Z 缓冲区的终止层级	屏幕上的 Hi-Z 追踪结果发生相应改变	正常
8	投射光线数量	调整 rayNums 的数值	渲染结果相应发生改变，rayNums 数值越高，噪点水平越低，渲染耗时越长，帧数越低；反之噪点水平越高，渲染耗时越短，帧数越高。	正常
9	光线步进最大步数	调整 raySteps 的数值	渲染结果相应发生改变，raySteps 数值越高，精度越高，渲染耗时越长，帧数越低；反之精度越低，渲染耗时越短，帧数越高。	正常
10	半分辨率追踪	调整 rayCastingResolution 为 Half	渲染结果相应发生改变，精度降低，但渲染耗时降低，帧数增加	正常
11	时域过滤	开启 Temporal Filter	噪点水平下降，变化速率降低，渲染耗时增加，帧数增加	正常
12	时域过滤的权重	调整 TemporalWeight 数值	渲染结果相应发生改变，噪点水平升高，渲染耗时和帧数不改变	正常
13	双边过滤	开启 Bilateral Filter	噪点水平下降，渲染耗时增加，帧数增加	正常
14	场景厚度	调整 thickness 数值	渲染结果相应发生改变，数值越高，整体场景越亮；数值越低，整体场景越暗，遮蔽感更强。渲染耗时和帧数不改变	正常

续表 6.2 系统功能性测试用例

序号	测试内容	测试方法	预期结果	测试结果
15	屏幕渐隐	调整 screenFade 数值	渲染结果相应发生改变，数值越高，屏幕边缘间接光比重越小；数值越高，屏幕边缘间接光比重越高。	正常
16	GI 强度	调整 giIntensity	渲染结果相应发生改变，数值越高，间接光照越不明显，场景越暗；数值越高，间接光照越明显，场景越亮	正常
17	UI 弹出与收回	点击 Tab 按键，点击弹出按钮	UI 界面正常弹出	正常
18	UI 控制开启 SSGI	点击启用 SSGI 选项	场景中出现间接漫反射光照，其他后处理不受影响	正常
19	UI 控制关闭 SSGI	点击禁用 SSGI 选项	场景中的间接漫反射光照消失，其他后处理不受影响	正常
20	切换 Debug Pass	点击下拉框切换 Debug Pass	渲染结果发生改变，根据旋转的下拉选项变成 Hi-Z 缓冲区或者 Indirect Color	正常
21	调整光照方向	拖动控制光照方向的滑动条	方向光的方向相应发生改变	正常
22	调整对比显示比例	拖动控制显示比例的滑动条	屏幕中的 SSGI 比例相应改变	正常
23	场景切换	点击数字按键 1-4	场景正常切换和加载	正常
24	创建并更换配置文件	在编辑器中右键创建配置文件，并将其赋值	场景根据配置文件的渲染参数正常的渲染	正常

如表 6.2 所示，对于本方法的各项功能模块如数据准备、加速结构、光线步进和降噪处理等部分，测试结果均符合预期，各个功能运行良好。

6.4 健壮性测试

SSGI 在场景中实时运行并计算间接光照，因此其稳定性和健壮性对于应用开发来说尤其重要。本部分对于运行和开发过程中可能出现的极端情况进行了测试模拟，以验证本方法的健壮性，测试用例如图 6.3 所示：

表 6.3 系统健壮性测试用例

序号	测试内容	测试方法	预期结果	测试结果
1	强制关闭 SSGI	运行时强制关闭 SSGI	场景中的间接光照效果消失，其他渲染效果和后处理效果不变	正常
2	配置文件丢失	销毁已创建的配置文件	使用默认参数的配置文件，渲染正常	正常

续表 6.3 系统健壮性测试用例

序号	测试内容	测试方法	预期结果	测试结果
3	渲染质量过高	将配置文件中的渲染质量调整到较高水平	UI 界面正常弹出	正常
4	场景材质丢失	删除物体上的材质组件	物体显示为出错（粉红色）材质	正常
5	操作撤销	对配置文件属性调整进行撤销	正确撤销	正常
6	更换配置文件	运行时更换配置文件	正确更换，渲染正常	正常

6.5 性能测试

SSGI 对直接光照的 Sponza 场景的全局光照效果如图 6.2 所示：

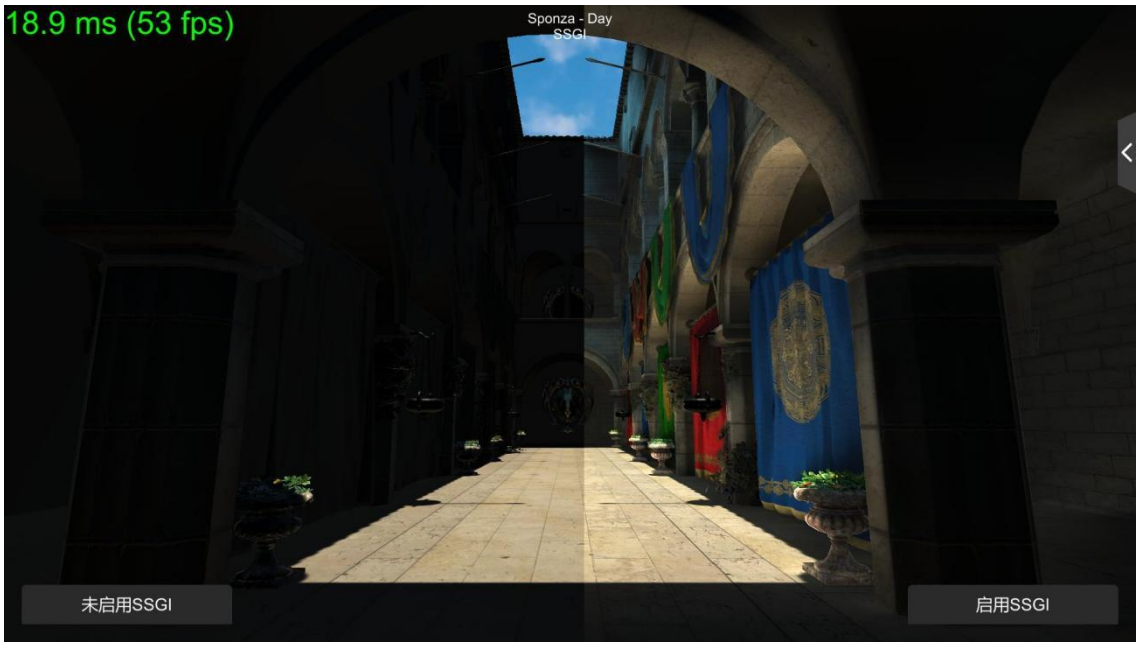

图 6.2 Sponza 场景中的 SSGI

主要的渲染质量有 rayNums = 8，Steps=32，该视角下的三角形面片数量为 1.8M，且由于该场景中存在大量实时布料结算（场景中的红绿蓝色布料），对 CPU 带来了额外的负担，对整体基准帧率带来了一定的影响。左侧为未启用 SSGI 的部分，由于场景中只有一盏方向光，且没有其他来源的间接光照，因此在没有直接光照的地方均为黑色；右侧是启用 SSGI 的部分，场景中除了方向光的直接光照之外，还有 SSGI 提供的间接光照，因此右侧的布料和墙壁都被间接光照所照亮。在被照亮的部分也会根据自身周围的几何遮挡产生不同的明暗关系，且在一定范围内产生了 Color Bleeding 的效果，整体光照的质量还是很高的。该渲染配置中默认打开了两种降噪手段，可以看到整体的噪点水平降低到了可接受的范围，且运行效率良好，整体帧率可以维持在较高水平。

除此之外，在测试测试机器上对其进行了性能测试。测试在 1920x1080 分辨率下进行，并打开时域过滤和双边过滤两种降噪手段，具体测试条件和结果如表 6.4 所示：

表 6.4 性能测试

序号	SSGI 是否开启	光线数量	最大光线步进次数	光线追踪分辨率	SSGI 平均渲染耗时（ms）	平均帧数
1	否					95
2	是	4	16	Full	1.2	75
3	是	4	32	Full	1.8	70
4	是	8	16	Full	1.7	62
5	是	8	32	Full	2.1	50
6	是	16	32	Full	2.5	35
7	是	16	64	Full	2.9	23
8	是	16	64	Half	2.6	26
9	是	32	64	Full	3.7	12
10	是	32	64	Half	3.1	14

其中序号 1 的测试条件为标准参数，即不开启 SSGI，场景中仅有方向光的直接光照时的帧数，当光线数量越高，最大光线步进次数越高的时，SSGI 渲染耗时会逐步增加，平均帧数会越来越低，同时开启半分辨率追踪可以提高 3-5 帧。在具体应用场景下，应该目标平台的性能和画面需求，来自定义渲染质量，一般认为，光线数量至少为 4，最大光线步长至少为 16 才能保证最基础的间接漫反射光照质量，过低的渲染参数就失去提升画面质量的意义了。在当前 Sponza 场景中，序号 5 的渲染参数可以同时兼顾应用流畅性和画面质量，如需更高的渲染质量，可以选择序号 6 或者序号 8 的渲染参数。

6.6 本章小结

本章通过在中低性能平台上进行功能性和健壮性测试，确保了各部分功能运行良好，同时具有一定的稳定性。在第五节中给出了具体的渲染效果以及不同参数下的性能表现。

7 总结与展望

7.1 工作总结

本文在屏幕空间相关 GI 算法的基础上，探索了在屏幕空间中使用实时光线追踪来计算场景中的间接漫反射光照，从而提供高质量 GI 效果的方法。

在方法设计方面，本文首先是对基于屏幕空间的相关算法和基于光线追踪的 GI 算法等进行了调研与研究。尤其参考了 SSR 的相关理论和实现，因此考虑将两者的优点结合起来，在屏幕空间中对场景几何信息进行简化，并使用 Hi-Z 缓冲区加速的光线步进来模拟光线与场景的求交过程，再对其进行相应的降噪处理，最终获得一个高质量的间接漫反射光照。

在具体实现方面，本文依托 Unity 引擎内置渲染管线中的延迟渲染路径，对方法进行了实现与测试，同时参考了相关商用渲染引擎中的功能模块，主要包括了纹理与矩阵数据准备、Hi-Z 缓冲区构建、屏幕空间光线步进、降噪处理等功能，并提供了一套用于测试的场景和用户交互模块。

在系统测试中，在 Sponza 场景中对其进行了相关功能性测试，并对几种边界和极端条件进行了健壮性测试，整体运行良好。最后给出了具体全局光照效果和各个渲染质量下的性能表现，整体效率良好，适合在中低性能的硬件平台上运行。

7.2 工作展望

屏幕空间 GI 算法的一大优势就在于其计算复杂度和场景复杂度无关，因此运行效率较高，且在延迟渲染中的 G Buffer 中，可以获取很多的场景信息。但也正因为屏幕空间中的场景几何信息不完整，因此其计算的间接光照效果是不准确的，仅仅是对间接光照的近似模拟。如果 SSGI 是场景中唯一的间接光照来源，那么在当光源和亮表面进入或者离开视野的时，场景中的全局光照效果会出现一定变化，即使使用了屏幕边缘渐隐等方法来减弱这种视觉变化，但是在部分视角和场景下仍然能够感受到场景中的光照变化，这也是所有屏幕空间 GI 算法无法避免的。

但是屏幕空间算法的另一优势在于具有很强的兼容性，它往往是作为后处理特效的一种，因此和其他屏幕空间效果一样，SSGI 最好是与其他间接光照技术结合使用，例如基于预计算的相关 GI 技术。将 SSGI 和烘焙光照叠加使用可以减少屏幕空间算法的瑕疵，因此和一些商用游戏引擎中一样，建议将 SSGI 用来改善场景中的间接光照而不是唯一的间接光照。

在本文方法的实现中，虽然最主要的光线步进等部分是借助 GPU 加速渲染计算的，但是纹理等资源的创建与传输都给 CPU 带来了额外的负担，尤其是 CPU 与 GPU 之间的通信受到带宽等因素的影响。如何进一步降低通信消耗，将大部分逻辑放在 GPU 执行，使得渲染过程完全由 GPU Driven 仍然需要进一步探索和研究。

另一方面， 随着软硬件技术的进一步发展，基于光栅化着色，并使用实时光线追踪来计算阴影和全局光照的混合渲染管线（Hybrid Render Pipeline）会成为未来一段时间的主流。

参考文献

[1] Akenine-Moller, Tomas, Eric Haines, and Naty Hoffman. Real-time rendering [M]. Fourth Edition. New York: AK Peters/CRC Press, 2018:437-511.

[2] Appel A. Some techniques for shading machine renderings of solids[C]//Proceedings of the April 30--May 2, 1968, spring joint computer conference. 1968: 37-45.

[3] Kajiya J T. The rendering equation[C]//Proceedings of the 13th annual conference on Computer graphics and interactive techniques. 1986: 143-150.

[4] 毛星云：实时光线追踪技术：业界发展近况与未来挑战[EB/OL]:https://zhuanlan.zhihu.com/p/102397700.Retrieved 2022.5.19.

[5] Sabino T L, Andrade P, Gonzales Clua E W, et al. A hybrid GPU rasterized and ray traced rendering pipeline for real time rendering of per pixel effects[A].International Conference on Entertainment Computing[C]. Springer, Berlin, Heidelberg, 2012: 292-305.

[6] 丁晓彤. 实时渲染中的动态全局照明技术[D]. 哈尔滨工业大学,2020.DOI:10.27061/d.cnki.ghgdu.2020.002220.

[7] Green R. Spherical harmonic lighting: The gritty details[C]//Archives of the game developers conference. 2003, 56: 4.

[8] Bavoil L, Sainz M. Multi-layer dual-resolution screen-space ambient occlusion[M]//SIGGRAPH 2009: Talks. 2009: 1-1.

[9] Beug A P. Screen Space Reflection Techniques[D]. Faculty of Graduate Studies and Research:University of Regina, 2020.

[10] Kaplanyan A, Dachsbacher C. Cascaded light propagation volumes for real-time indirect illumination[C]//Proceedings of the 2010 ACM SIGGRAPH symposium on Interactive 3D Graphics and Games. 2010: 99-107.

[11] Panteleev A.NVIDIA VXGI: Dynamic Global Illumination for Games[R]. GDC Presentation,2015:23-25.

[12] Greene N, Kass M, Miller G. Hierarchical Z-buffer visibility[C]//Proceedings of the 20th annual conference on Computer graphics and interactive techniques. 1993: 231-238.

[13] Akenine-Moller, Tomas, Eric Haines, and Naty Hoffman. Real-time rendering [M]. Fourth Edition. New York: AK Peters/CRC Press, 2018:11-27.

[14] Microsoft：顶点着色器阶段[EB/OL]:https://docs.microsoft.com/zh-cn/windows/win32/direct3d11/vertex-shader-stage.Retrieved 2022.5.19.

[15] Jimenez J, Gutierrez D, Yang J, et al. Filtering approaches for real-time anti-aliasing[J].

SIGGRAPH Courses, 2011, 2(3): 4.

[16] Microsoft：像素着色器阶段[EB/OL]:https://docs.microsoft.com/zh-cn/windows/win32/direct3d11/pixel-shader-stage.Retrieved 2022.5.19.

[17] Shishkovtsov O. Deferred shading in stalker[J]. Gpu gems, 2005, 2: 143-166.

[18] 洛城.游戏引擎中的渲染管线[EB/OL].https://zhuanlan.zhihu.com/p/92165837. Retrieved 2021-5-19.

[19] T. Saito and T. Takahashi, 'Comprehensible rendering of 3-D shapes', ACM SIGGRAPH Computer Graphics, vol. 24, no. 4, pp. 197-206, 1990.

[20] Frozen Claw. 延迟渲染与MSAA的那些事 [EB/OL]. https://zhuanlan.zhihu.com/p/135444145.Retrieved 2022.5.19.

[21] insulation：insulation[EB/OL]:https://zhuanlan.zhihu.com/p/432910255.Retrieved 2022.5.19.

[22] Bavoil L, Sainz M, Dimitrov R. Image-space horizon-based ambient occlusion[A]. ACM SIGGRAPH 2008 talks[C], Los Angeles California: ACM , 2008: 1-1.

[23] Jiménez J, Wu X, Pesce A, et al. Practical real-time strategies for accurate indirect occlusion[J]. SIGGRAPH 2016 Courses: Physically Based Shading in Theory and Practice, 2016.

[24] Ritschel T, Grosch T, Seidel H P. Approximating dynamic global illumination in image space[A].Proceedings of the 2009 symposium on Interactive 3D graphics and games[C], Boston Massachusetts: ACM, 2009: 75-82.

[25] Screen-Space Reflections:Stochastic Screen-Space Reflections[EB/OL]: https://www.ea.com/frostbite/news/stochastic-screen-space-reflections.Retrieved 2022.5.19.

[26] ERICPOLMAN:Light Propagation Volumes[EB/OL]: https://ericpolman.com /2016/06/28/ light-propagation-volumes.Retrieved 2022.5.19

[27] Nvidia:VSGI[EB/OL]:https://www.nvidia.cn/geforce/technologies/vxgi/technology/.Retrieved 2022.5.19.

[28] Batchelor, James (March 20, 2018). "Unity 2018 detailed in GDC keynote".gamesindustry.biz. Archived from the original on March 20, 2019.

[29] Dubois E, Sabri S. Noise reduction in image sequences using motion-compensated temporal filtering[J]. IEEE transactions on communications, 1984, 32(7): 826-831.

[30] 默默默默默：[GDC Talk]低复杂，高保真——INSIDE里的渲染技术[EB/OL]: https://zhuanlan.zhihu.com/p/161367110.Retrieved 2022.5.19.

[31] 董玉德,汪玉玺,刘达新,王万龙.三角平面Halton点采样策略及其性能分析[J].计算机

辅助设计与图形学学报,2007(08):1063-1068.

[32] 张志强,王万玉.一种改进的双边滤波算法[J].中国图象图形学报,2009,14(03):443-447.

[33] McGuire M, Mara M. Efficient GPU screen-space ray tracing[J]. Journal of Computer Graphics Techniques (JCGT), 2014, 3(4): 73-85.

[34]]朱陆陆. 蒙特卡洛方法及应用[D].华中师范大学,2014.

[35] Unity Technology Motion Vectors[EB/OL]:https://docs.unity3d.com/ScriptReference/DepthTextureMode.MotionVectors.html.Retrieved 2022.5.19.

[36] Granados M, Ajdin B, Wand M, et al. Optimal HDR reconstruction with linear digital cameras[C]//2010 IEEE computer society conference on computer vision and pattern recognition. IEEE, 2010: 215-222.

[37] Tomasz Stachowiak Stochastic Screen-Space Reflections[EB/OL]:Stochastic Screen-Space Reflections - Frostbite (ea.com).

[38] Shirley P, Chiu K. A low distortion map between disk and square[J]. Journal of graphics tools, 1997, 2(3): 45-52.

[39] MaxwellGeng:Unity Temporal AA 的改进与提高 [EB/OL]: https://zhuanlan.zhihu.com/p/46841906.Retrieved 2022.5.19

[40] JPG:Notes On Screen Space HiZ Tracing[EB/OL]: https://www.jpgrenier.org/ssr.html .Retrieved 2022.5.19.

致 谢

在大三之前，其实我并不清楚自己未来的方向，只是凭借着对游戏的热爱，按照学校设置的相关课程进行被动的学习，后来了解到和图形学相关的引擎程序员与图形程序员是游戏行业中的“高端工种”，因此才对图形算法产生了较强的兴趣。后来在谷祖安学长的建议下（一起打游戏时），我选择尝试完成一个屏幕空间的 GI 算法作为自己的毕业设计。

在大学四年的时间中，我首先需要感谢我亲爱的父母，有些辛酸只有长大了才能体会，才能理解。我对母亲在患难中不抛弃不放弃的精神感到深深的敬佩，她是一个多么伟大的朴素的勤劳的女性啊！在读完研究生之后，我会努力担起家庭的担子，让母亲不再忧虑。

在学术方面，非常感谢我的指导老师高天寒教授，高老师在毕设期间给了我非常多的帮助。在毕设推进的关键时期，高老师坚持开展线上例会，一方面保证了毕设的进度，另一方面也指导我突出工作中的亮点；在毕业论文撰写阶段，老师制订了详细的论文进度时间点，并分享了丰富且实用的论文书写经验与技巧。高老师的细致指导使得我在毕业设计期间获益良多，并最终完成了令人满意的成果。在此，再次表达我对高天寒老师的由衷的感谢与敬意。

感谢我的室友以及实验室的学长学姐，感谢他们的一路陪伴，难忘那些快乐的青春时光。感谢我的母校和各位任课老师，在四年的学习生活中教会了我各种专业知识，为今后的研究生学习打下了坚实的基础。

最后需要感谢党和国家，作为一名预备党员，我坚信怀有共产主义信仰，在历史唯物主义指导下的新时代中国青年，能够走好属于我们这一代人的长征路，太阳会照常升起！